

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

МЕТОДИЧНІ ВКАЗІВКИ
до виконання самостійних робіт
з дисципліни «Безпека та захист програм і даних»

для студентів спеціальності
121 «Інженерія програмного забезпечення»
усіх форм навчання

2023

Методичні вказівки до виконання самостійних робіт з дисципліни «Безпека та захист програм і даних» для студентів спеціальності 121 «Інженерія програмного забезпечення» усіх форм навчання /Укл.: Т.А.Зайко, О.І.Качан. – Запоріжжя: НУ «Запорізька політехніка», 2023. – 70 с.

Укладачі: Т.А.Зайко, к.т.н., доцент кафедри ПЗ,
О.І.Качан, старший викладач кафедри ПЗ.

Рецензент: А.О.Олійник, д.т.н., професор кафедри ПЗ.

Відповідальний
за випуск: С.О.Субботін, д.т.н., професор кафедри ПЗ.

Затверджено
на засіданні кафедри
«Програмні засоби»
Протокол № 12
від «09» червня 2023 р.

Рекомендовано до видання
НМК Факультету
комп'ютерних наук і технологій
Протокол № 1
від «30» серпня 2023 р.

ЗМІСТ

ЗАГАЛЬНІ ПОЛОЖЕННЯ	5
1 ЛАБОРАТОРНА РОБОТА №1	
БАЗОВІ ШИФРИ. ЧАСТОТНИЙ КРИПТОАНАЛІЗ.....	6
1.1 Короткі теоретичні відомості.....	6
1.2 Завдання на лабораторну роботу.....	12
1.3 Зміст звіту.....	12
1.4 Контрольні питання.....	12
2 ЛАБОРАТОРНА РОБОТА №2	
РЕЖИМИ ШИФРУВАННЯ БЛОКОВИХ ШИФРІВ	13
2.1 Короткі теоретичні відомості	13
2.2 Завдання на лабораторну роботу.....	20
2.3 Зміст звіту.....	21
2.4 Контрольні питання.....	21
3 ЛАБОРАТОРНА РОБОТА №3	
СТЕГАНОГРАФІЧНИЙ ЗАХИСТ	22
3.1 Короткі теоретичні відомості	22
3.2 Завдання на лабораторну роботу.....	27
3.3 Зміст звіту.....	27
3.4 Контрольні питання.....	27
4 ЛАБОРАТОРНА РОБОТА №4	
ДОСЛІДЖЕННЯ РОБОТИ ПРОГРАМ-МОНІТОРІВ	28
4.1 Теоретичні відомості.....	28
4.2 Завдання на лабораторну роботу.....	34
4.3 Зміст звіту.....	34
4.4 Контрольні питання.....	35
4.5 Порядок виконання роботи	35
5 ЛАБОРАТОРНА РОБОТА №5	
ЗАХИСТ ПРОГРАМ ЗА ДОПОМОГОЮ ПРОГРАМ-	
ПАКУВАЛЬНИКІВ.....	37
5.1 Теоретичні відомості.....	37
5.2 Завдання на лабораторну роботу.....	46
5.3 Зміст звіту.....	46
5.4 Контрольні питання.....	46
5.5 Порядок виконання роботи	46

6 ЛАБОРАТОРНА РОБОТА №6	
ДОСЛІДЖЕННЯ СТРУКТУРИ ВИКОНУВАНИХ ФАЙЛІВ.....	48
6.1 Стислі теоретичні відомості	48
6.2 Завдання на лабораторну роботу.....	53
6.3 Зміст звіту.....	53
6.4 Контрольні питання.....	54
6.5 Порядок виконання роботи	54
7 ЛАБОРАТОРНА РОБОТА №7	
РОЗРОБКА І РЕАЛІЗАЦІЯ ВБУДОВАНОГО ЗАХИСТУ	
ПРОГРАМ.....	55
7.1 Теоретичні відомості	55
7.2 Завдання на лабораторну роботу.....	62
7.3 Зміст звіту.....	62
7.4 Контрольні питання.....	62
7.5 Порядок виконання роботи	63
ПЕРЕЛІК ПОСИЛАНЬ	65
ДОДАТОК А	
ТАБЛИЦЯ ПРОСТИХ ЧИСЕЛ	66
ДОДАТОК Б	
ПІДКЛЮЧЕННЯ КРИПТОГРАФІЧНОЇ БІБЛІОТЕКИ MIRACL	
.....	67

ЗАГАЛЬНІ ПОЛОЖЕННЯ

Методичні вказівки є необхідним засобом для успішного вивчення теоретичного матеріалу та практичного засвоєння дисципліни «Безпека та захист програм і даних».

Мета роботи – підвищити рівень знань студентів у галузі криптографії та шифрування даних, а також познайомити їх з методиками захисту програм від стороннього втручання, вивчити алгоритмічну структуру програмних модулів та відлагоджувачів на мікропрограмному рівні. До складу лабораторного курсу належать сім робіт, які охоплюють необхідну галузь криптографії, а також загальні принципи та засоби захисту програм і даних від стороннього втручання. Кожна робота виконується та здається студентом індивідуально. Студенти, що не підготовлені до роботи, а також, які не мають вірно оформленого звіту, до занять не допускаються.

Вимоги до оформлення та змісту звіту, а також контрольні запитання, представлені в кожній лабораторній роботі. Студент повинен знати мету роботи, теоретичні відомості, методику побудови та відлагодження необхідних програм.

1 ЛАБОРАТОРНА РОБОТА №1

БАЗОВІ ШИФРИ. ЧАСТОТНИЙ КРИПТОАНАЛІЗ

Мета роботи: ознайомитися з базовими шифрами. Розглянути методику частотного криптоаналізу.

1.1 Короткі теоретичні відомості

В криптографії здавна використовувались два види шифрів: заміна та перестановка. Історичним прикладом шифру заміни є шифр Цезаря. Його сутність така: в строку виписується алфавіт, після чого під ним виписується той же алфавіт з циклічним зсувом на 3 букви ліворуч.

АБВГДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯ
ГДЕЄЖЗИІЇЙКЛМНОПРСТУФХЦЧШЩЬЮЯБВ

У процесі шифрування буква початкового тексту (яка береться з верхнього рядку) замінюється на букву, що знаходиться під нею в нижньому рядку. Наприклад: **РИМ** – **УЙП**. Ключем у шифрі Цезаря є величина зсуву нижнього рядка, тобто 3.

Для дешифрування, або отримання оригінального початкового тексту, необхідно знайти у другому рядку букву із зашифрованого тексту та виконати заміну її на відповідну букву із першого рядка.

1.1.1 Шифр простої заміни

Подальший розвиток шифру Цезаря є зрозумілим: нижній рядок може бути створено з випадковим порядком букв алфавіту. Такий шифр носить узагальнену назву шифру простої заміни. Ключем такого шифру є порядок розташування букв у нижньому рядку, – це має назву «таблиця заміни». Якщо в шифрі Цезаря існує тільки 32 варіанти ключів (за кількістю букв алфавіту), то в шифрі простої заміни їх вже 32! (факторіал від 32, за всіма варіантами розташування букв алфавіту, включно оригінальний початковий порядок букв).

1.1.2 Квадрат Полібія

Одна з відомих модифікацій шифру простої заміни – квадрат Полібія. Візьмемо алфавіт з 32 букв. Оберемо ключ – будь-яке слово, в якому немає однакових букв. Запишемо його в перші клітинки квадрата розміром, наприклад, 4×8. Далі в наступні вільні клітинки

запишемо букви алфавіту за винятком тих букв, що зустрічаються в ключі. Для шифрування букви початкового тексту повідомлення замінюються на букви, що знаходяться під ними в квадраті Полібія. Якщо буква початкового тексту випадає на останній рядок, то вона буде заміненена на букву першого рядку. Наприклад:

Початкове повідомлення – «ДУМИ МОЇ, ДУМИ МОЇ, ЛИХО МЕНІ З ВАМИ! НАЩО СТАЛИ НА ПАПЕРІ СУМНИМИ РЯДАМИ? . . .».

Ключ – «ДНІПРО».

На рисунку 1.1 надано Квадрат Полібія для ключа «ДНІПРО».

Д	Н	І	П
Р	О	А	Б
В	Г	Е	Є
Ж	З	И	Ї
Й	К	Л	М
С	Т	У	Ф
Х	Ц	Ч	Ш
Щ	Ъ	Ю	Я

Рисунок 1.1 – Квадрат Полібія

У початковому повідомленні виключаємо знаки пунктуації та роздільники слів:

«ДУМИМОЇДУМИМОЇЛИХОМЕНІЗВАМИНАЩОСТАЛИ
НАПАПЕРІСУМНИМИРЯДАМИ».

Зашифроване повідомлення:

«РЧФЛФГМРЧФЛФГМУЛЩГФИОАКЖЕФЛОЕДГХЦЕУЛ
ОЕБЕБИВАХЧФОЛФЛВПРЕФЛ».

У процесі розшифровування букви шифротексту замінюються згідно даного шифру для заданого ключа на ті букви, що стоять над ними в квадраті Полібія.

1.1.3 Шифр перестановки

Шифр перестановки відноситься до відокремленого виду шифрів. Має алгоритмічну простоту, незалежність від елементів алфавіту, а також величезну криптографічну стійкість. Тому, навіть, і

сьогодні, найсучасніші шифрувальні системи, майже кожні, мають у своїх алгоритмах декілька блоків, заснованих на шифрі перестановки.

Суть шифру перестановки є в заміні одних елементів оригінального початкового тексту другими елементами того ж початкового тексту. Якщо обрати ціле додатне число, наприклад, 5, яке буде вказувати на довжину ключа, потім випадковим методом перетасувати числа від 1 до 5 та внести значення в другий рядок таблиці (рис. 1.2), отримаємо наступну підстановочну таблицю:

Початкова позиція символу	1	2	3	4	5
Змінена позиція символу	4	3	2	5	1

Рисунок 1.2 – Таблиця шифру перестановки

Наприклад:

Початкове повідомлення – «СЕЛО НЕНАЧЕ ПОГОРІЛО, НЕНАЧЕ ЛЮДИ ПОДУРІЛИ, НІМІ НА ПАНЩИНУ ІДУТЬ І ДІТОЧОК СВОЇХ ВЕДУТЬ!...».

Ключ – «43251», – другий рядок таблиці (рис. 1.2).

У початковому повідомленні виключаємо знаки пунктуації та роздільники слів:

«СЕЛОНЕНАЧЕПОГОРІЛОНЕНАЧЕЛЮДИПОДУРІЛИ
НІМІНАПАНЩИНУІДУТЬІДІТОЧОКСВОЇХВЕДУТЬ».

Для шифрування повідомлення доповнимо фразу до довжини кратної 5 випадковими символами та розіб'ємо на групи по 5 букв:

«СЕЛОН ЕНАЧЕ ПОГОР ІЛОНЕ НАЧЕЛ ЮДИПО
ДУРІЛ ІНІМІ НАПАН ЩИНУІ ДУТЬІ ДІТОЧ
ОКСВО ЇХВЕД УТЬХІ».

Букви кожної групи переставимо згідно обраної підстановки:

«ОЛЕНС ЧАНЕЕ ОГОРП НОЛЕІ ЕЧАЛН ПИДОЮ
ІРУЛД МІНІІ АПАНН УНІЩ ЪТУІД ОТИЧД
ВСКОО ЕВХДІ ХЬТІУ».

Отриманий текст розіб'ємо на групи по 7 букв для зміщення акценту та доповнимо випадковими символами останню групу:

«ОЛЕНСЧА НЕЕОГОР ПНОЛЕІЕ ЧАЛНПИД
ОЮІРУЛД МІНІІАП АННУНІ ЩЬТУІДО ТІЧДВСК
ООВЕХДІ ХЬТІУНГ».

При дешифруванні текст розбивається на групи довжиною ключа (5 букв), а потім букви переставляються у зворотньому порядку.

1.1.4 Шифр Тритемія

У XV столітті абат Тритемій (Германія) запропонував шифр на основі «Таблиці Тритемія». Для українського алфавіту вона має наступний вигляд (рис. 1.3).

А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	Я	
Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Я	
В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	Я	
Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Я	
Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Я	
Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Я	
Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Я	
З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	Я	
И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	Я	
І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	Я	
Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Я	
К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	Я	
Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Я	
М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	Я	
Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Я	
О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	Я	
П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	Я	
Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Я	
С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	Я	
Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Я	
У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	Я	
Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Я	
Х	Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Я	
Ц	Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Я	
Ч	Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Я	
Ш	Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Я	
Щ	Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Я	
Ь	Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Я	
Ю	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Я	
Я	А	Б	В	Г	Д	Е	Ж	З	И	І	Й	К	Л	М	Н	О	П	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ь	Ю	Я

Рисунок 1.3 – Таблиця Тритемія для українського алфавіту

Перший рядок таблиці є одночасно рядком букв початкового тексту. Перша буква тексту шифрується по першому рядку, друга – по другому і т.д. Після останнього рядка у таблиці знову повертаємось до першого. Наприклад, слово “КРИПТОГРАФІЯ” зміниться на “КСІТЦУИЧЗАСІ”. Однак, у такому варіанті, в шифрі Третеція був відсутній ключ. У подальшому удосконаленні шифру пішли по двом шляхам:

- введення випадкового порядку розташування букв алфавіту;
- застосування більш складного порядку вибору рядків таблиці по ключу (розширення до шифру Віжинера).

1.1.5 Частотний криптоаналіз

Частотний криптоаналіз базується на застосуванні статистики для аналізу текстової інформації. Текст складається із слів, слова із літер. Кількість літер у кожній мові обмежена. Важливими характеристиками тексту є повторюваність літер, пар літер (біграм), взагалі, m-грам, поєднання літер друг с другом, чередування голосних і приголосних та інше. Всі ці характеристики є достатньо стійкими і можуть бути використані для аналізу шифротекстів. У табл. 1.1 та 1.2 наведено однобуквені ймовірності англійської та української мов.

Таблиця 1.1 – Однобуквені ймовірності англійської мови

Літера	Ймовірність	Літера	Ймовірність
A	0.0856	N	0.0707
B	0.0139	O	0.0797
C	0.0279	P	0.0199
D	0.0378	Q	0.0012
E	0.1304	R	0.0677
F	0.0289	S	0.0607
G	0.0199	T	0.1045
H	0.0528	U	0.0249
I	0.0627	V	0.0092
J	0.0013	W	0.0149
K	0.0042	X	0.0017
L	0.0339	Y	0.0199
M	0.0249	Z	0.0008

Таблиця 1.2 – Однобуквені ймовірності української мови

Літера	Ймовірність	Літера	Ймовірність
А	0.072	Н	0.065
Б	0.017	О	0.094
В	0.052	П	0.029
Г	0.016	Р	0.047
Д	0.035	С	0.041
Е	0.017	Т	0.055
Є	0.008	У	0.040
Ж	0.009	Ф	0.001
З	0.023	Х	0.012
И	0.061	Ц	0.006
І	0.057	Ч	0.018
Ї	0.006	Ш	0.012
Й	0.008	Щ	0.001
К	0.035	Ь	0.029
Л	0.036	Ю	0.004
М	0.031	Я	0.029

Процес криптоанализу можна представити наступним чином. Криптоаналітик підраховує частоти символів у шифротексті. Надалі ця множина символів відсортовується за частотою від найбільшої до найменшої. Потім виконується співставлення цієї множини з відсортованою таким же чином множиною символів (букв) передбачуваного алфавіту. Далі проводиться пошук найближчих сусідів обох множин до однієї величини. Припускається, що ці обидва найближчих сусіда є однією буквою цільового алфавіту. Шляхом проб і помилок такий метод може привести до рішення задачі. Крім того, при підставленні букв замість символів аналізованого шифротексту криптоаналітик враховує частоти появи сполучень із двох букв (біграм), трьох букв (триграм) та інших.

1.2 Завдання на лабораторну роботу

1.2.1 Розробити програми шифрування та дешифрування одним з наступних шифрів на вибір викладача:

- шифр простої заміни;
- квадрат Полібія;
- шифр перестановки;
- шифр Тритемія;
- шифр Віжинера.

1.2.2 Виконати частотний криптоаналіз отриманих зашифрованих текстів.

1.3 Зміст звіту

1.3.1 Титульний лист, тема і мета роботи.

1.3.2 Відповіді на контрольні питання.

1.3.3 Тексти програм.

1.3.4 Обране повідомлення для шифрування.

1.3.5 Обраний ключ.

1.3.6 Зашифроване повідомлення.

1.3.7 Розшифроване повідомлення.

1.3.8 Результати проведення частотного криптоаналізу.

1.4 Контрольні питання

1.4.1 Опишіть шифр Полібія.

1.4.2 Опишіть шифр простої заміни.

1.4.3 Опишіть шифр Тритемія.

1.4.4 Опишіть шифр перестановки.

1.4.5 Чи є шифр Полібія шифром простої заміни?

1.4.6 Як залежить стійкість шифру від довжини ключа?

1.4.7 Опишіть метод частотного криптоаналізу.

1.4.8 В яких випадках можна застосовувати метод частотного криптоаналізу?

2 ЛАБОРАТОРНА РОБОТА №2

РЕЖИМИ ШИФРУВАННЯ БЛОКОВИХ ШИФРІВ

Мета роботи: ознайомитися з програмною реалізацією алгоритму симетричного блокового шифрування RIJNDAEL. Розробити власні програми його використання у різних режимах шифрування.

2.1 Короткі теоретичні відомості

2.1.1 Опис алгоритму шифрування RIJNDAEL

На цей час в Україні діє міждержавний стандарт симетричного шифрування ГОСТ 28147-89. У 1997 році національний інститут стандартів і технологій США (NIST) об'явив про початок програми щодо прийняття нового стандарту криптографічного захисту для закриття важливої інформації урядового рівня. Фіналістом конкурсу став RIJNDAEL, AES (Advanced Encryption Standard) – Федеральний стандарт шифрування США, затверджений міністерством торгівлі США як стандарт, 4 грудня 2001 року. Місце розробки – Бельгія, 1997 рік. Автори Йоан Дамен (Joan Daemen) та Винсент Раймен (Vincent Rijmen).

Алгоритм симетричного блокового шифрування RIJNDAEL становить особливий інтерес як новий американський стандарт криптографічного захисту – стандарт XXI століття для закриття важливої інформації урядового рівня, який прийшов на заміну існуючому з 1974 року алгоритму DES, найпоширенішому криптоалгоритму в світі. В алгоритмі RIJNDAEL не виявлено слабких місць у захисті.

Цей алгоритм відрізняють:

- висока ефективність на будь-яких платформах;
- високий рівень захищеності;
- шифр добре підходить для реалізації в smart-картах через низькі вимоги до пам'яті;
- швидка процедура формування ключа;
- ефективна підтримка паралелізму на рівні інструкцій;
- підтримка різних довжин ключа з кроком у 32 біта.

Параметри шифру RIJNDAEL наведені у табл. 2.1 та 2.2.

Таблиця 2.1 – Параметри шифру RIJNDAEL

Розмір блоку, біт	128, 192, 256
Розмір ключа, біт	128, 192, 256
Число раундів	10, 12, 14
Розмір ключового елемента, біт	128, 192, 256 (дорівнює розміру блоку)
Кількість ключових елементів	11, 13, 15 (на 1 більше числа раундів)

Кількість раундів шифрування залежить від розмірів ключа та блоку.

Таблиця 2.2 – Кількість раундів шифрування

Розмір ключа	128	192	256
Розмір блоку			
128	10	12	14
192	12	12	14
256	14	14	14

Один раунд шифрування на псевдокодi виглядає наступним чином:

```
Round (State, RoundKey) {
    ByteSub (State) ;
    ShiftRow (State) ;
    MixColumn (State) ;
    AddRoundKey (State, RoundKey) ;
}
```

Останній раунд шифрування на псевдокодi виглядає так:

```
Round (State, RoundKey) {
    ByteSub (State) ;
    ShiftRow (State) ;
    AddRoundKey (State, RoundKey) ;
}
```

У цьому криптоалгоритмі деякі операції виконуються над байтами, що розглядаються як елементи поля $GF(2^8)$. Елементами $GF(2^8)$ є двійкові багаточлени ступені $N < 8$, що можуть бути задані рядком своїх коефіцієнтів. При такому представленні елементів поля, додавання в полі $GF(2^8)$ – це операція порозрядного XOR, а множення – це звичайна операція множення багаточленів з узяттям результату по модулю деякого незвідного двійкового багаточлена $\phi(x)$, з використанням операції XOR при приведенні подібних членів. У RIJNDAEL обраний незвідний поліном $\phi(x) = x^8 + x^4 + x^3 + x + 1$ показника 51.

Блок даних представляється у вигляді масиву значень у табл. 2.3.

Таблиця 2.3 – Представлення блоків даних

a_{00}	a_{01}	a_{02}	a_{03}
a_{10}	a_{11}	a_{12}	a_{13}
a_{20}	a_{21}	a_{22}	a_{23}
a_{30}	a_{31}	a_{32}	a_{33}

У процедурі ByteSub: a_{ij} змінюється на a_{ij}^{-1} в полі Галуа $GF(2^8)$.

$$a_{ij} * a_{ij}^{-1} = 1 \pmod{(x^8 + x^4 + x^3 + x + 1)} \quad (2.1)$$

Афінне перетворення діє за правилом:

$$Y = C * x + C_1 \pmod{(x^8 + 1)} \quad (2.2)$$

$$\begin{array}{cccccccccc}
 y_0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & x_0 & 1 \\
 y_1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & x_1 & 1 \\
 y_2 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & x_2 & 0 \\
 y_3 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & x_3 & 0 \\
 y_4 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & x_4 & 0 \\
 y_5 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & x_5 & 1 \\
 y_6 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & x_6 & 1 \\
 y_7 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & x_7 & 0
 \end{array} \quad (2.3)$$

У процедурі ShiftRow останні три рядки стану циклічно зсуваються на різне число байтів (C1, C2, C3) у залежності від довжини блоку Nb (табл. 2.4).

Таблиця 2.4 – Зсув рядків

Nb	C1	C2	C3
4	1	2	3
6	1	2	3
8	1	3	4

У процедурі MixColumn стовпці стану розглядаються як багаточлени над $GF(2^8)$ та помножуються на багаточлен $g(x) = '03'x^3 + '01'x^2 + '01'x + '02'$ за модулем $(x^4 + 1)$

$$\begin{array}{cccccc}
 b_0 & 02 & 03 & 01 & 01 & a_0 \\
 b_1 & 01 & 02 & 03 & 01 & a_1 \\
 b_2 & 01 & 01 & 02 & 03 & a_2 \\
 b_3 & 03 & 01 & 01 & 02 & a_3
 \end{array} \quad (2.4)$$

Загальне число бітів раундових ключів дорівнює довжині блоку, помноженої на число раундів, плюс 1 (наприклад, для довжини блоку

128 біт і 10 циклів буде потрібно 1408 біт циклового ключа). Ключ шифрування розширюється в розширений ключ. Раундові ключі беруться з розширеного ключа в такий спосіб: перший раундовий ключ містить перші N_b слів, другий – наступні N_b слів і т.д.

2.1.2 Режими шифрування

Під режимом шифрування розуміється такий алгоритм застосування блокового шифру, що при відправленні повідомлення дозволяє перетворювати відкритий текст у шифротекст і, після передачі цього шифротексту по відкритому каналу однозначно відновити первісний відкритий текст. Як видно з визначення, сам блоковий шифр тепер є тільки частиною іншого алгоритму – алгоритму режиму шифрування. Це обумовлено тим, що блоковий шифр працює тільки з окремим блоком даних, у той час як алгоритм режиму шифрування має справу вже з цілим повідомленням, що може складатися з деякого числа n блоків. Більш того, повідомлення взагалі не зобов'язане складатися з блоків, у тім змісті, що це повідомлення не завжди можна розділити на ціле число n блоків. У цьому випадку в різних режимах шифрування потрібно доповнювати повідомлення різною кількістю біт.

Передбачено можливість функціонування блокових шифрів у наступних режимах шифрування:

- режим електронна кодова книга (ECB);
- режим зціплення блоків шифротексту (CBC);
- режим зворотнього зв'язку за шифротекстом (CFB);
- режим зворотнього зв'язку по виходу (OFB);
- режим шифрування з лічильником (Counter).

На рисунках 2.1 – 2.5 приведені схеми шифрування в цих режимах. Використані наступні позначення:

- P_i – i -й блок відкритого тексту;
- E_k – функція зашифрування на ключі k ;
- C_i – i -й блок зашифрованого тексту;
- D_k – функція розшифрування на ключі k ;
- S_i – i -те значення лічильника;
- S_0 – синхропосилка.

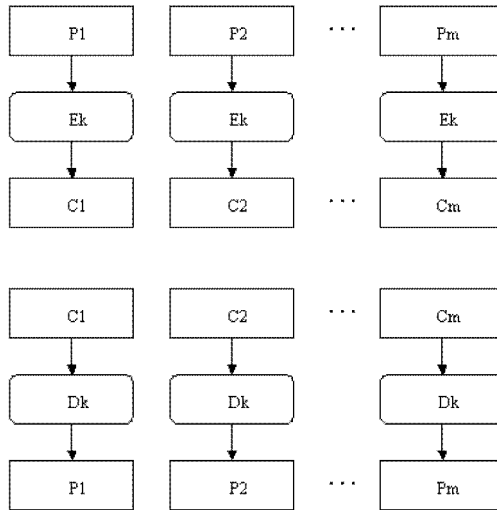


Рисунок 2.1 – Режим електронна кодова книга (ECB)

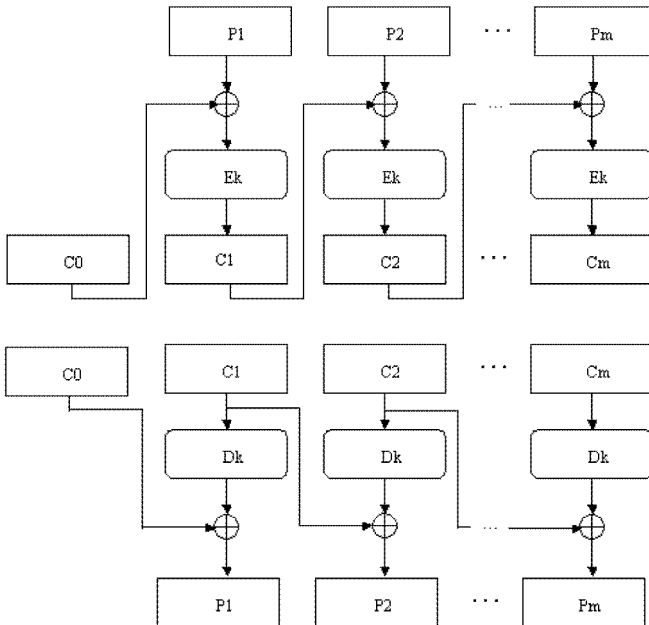


Рисунок 2.2 – Режим зіплення блоків шифротексту (CBC)

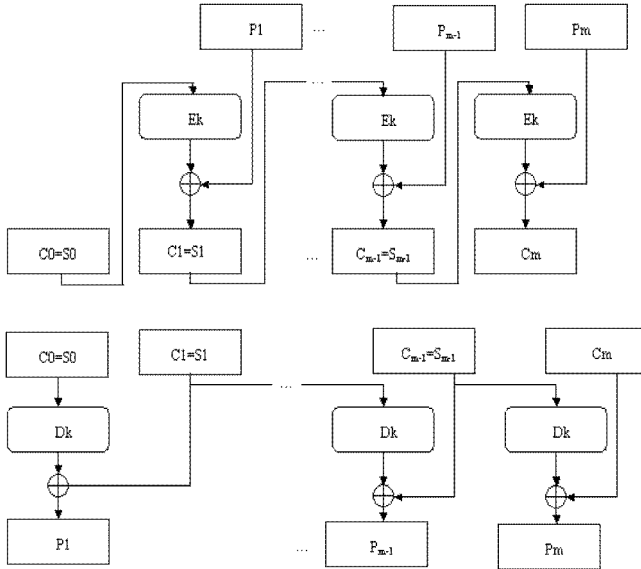


Рисунок 2.3 – Режим зворотного зв'язку за шифротекстом (CFB)

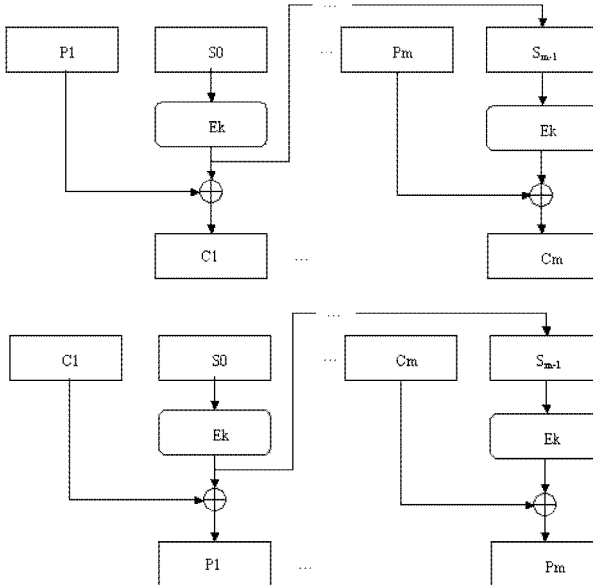


Рисунок 2.4 – Режим зворотного зв'язку по виходу (OFB)

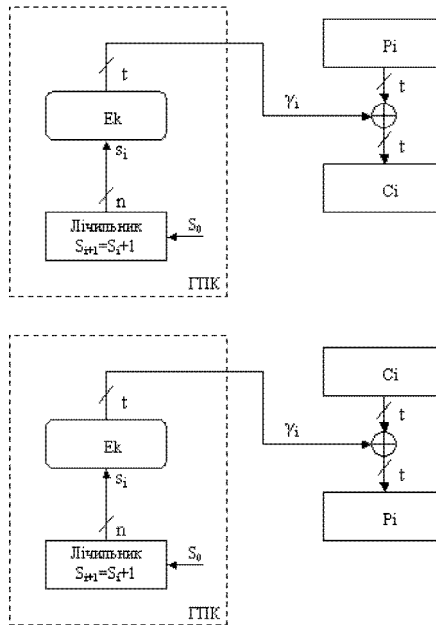


Рисунок 2.5 – Режим шифрування з лічильником (Counter)

Розглянуті режими шифрування представлені в документі Recommendation for Block Cipher Modes of Operation. NIST Special Publication 800-38A. Technology Administration U.S. Department of Commerce. 2001 Edition, що був виданий NIST США та має назву «Рекомендації для режимів шифрування з блоковим шифром».

2.2 Завдання на лабораторну роботу

2.2.1 Створити проект з підключенням бібліотеки MIRACL згідно додатку Б. Додати до проекту файл `mraes.c` з каталогу `\miracl\source`. Відкомпілювати та запустити програму. По тексту програми розібрати програмну реалізацію алгоритму симетричного шифрування AES (RIJNDAEL).

2.2.2 Розробити програму використання алгоритму RIJNDAEL у наступних режимах шифрування:

- режим електронна кодова книга (ECB);
- режим зціплення блоків шифротексту (CBC);

- режим зворотнього зв'язку за шифротекстом (CFB);
- режим зворотнього зв'язку по виходу (OFB);
- режим шифрування з лічильником (Counter).

2.3 Зміст звіту

- 2.3.1 Титульний лист, тема та мета роботи.
- 2.3.2 Відповіді на контрольні питання.
- 2.3.3 Опис функцій бібліотеки MIRACL для реалізації AES.
- 2.3.4 Тексти програм.
- 2.3.5 Результати роботи програм.
- 2.3.6 Висновки.

2.4 Контрольні питання

- 2.4.1 З яких операцій складається раунд алгоритму RIJNDAEL?
- 2.4.2 Поясніть режим електронна кодова книга (ECB).
- 2.4.3 Поясніть режим зціплення блоків шифротексту (CBC).
- 2.4.4 Поясніть режим зворотнього зв'язку за шифротекстом (CFB).
- 2.4.5 Поясніть режим зворотнього зв'язку по виходу (OFB).
- 2.4.6 Поясніть роботу алгоритму в режимі шифрування з лічильником (Counter).
- 2.4.7 Чому шифри ГОСТ 28147-89 та RIJNDAEL називаються блоковими шифрами?
- 2.4.8 Якими можуть бути довжина ключа та блока в ГОСТ 28147-89 та в RIJNDAEL?
- 2.4.9 В чому полягає основний шаг криптосистем ГОСТ 28147-89 та RIJNDAEL?
- 2.4.10 Перелічіть основні режими шифрування блокових шифрів.
- 2.4.11 Які основні параметри шифрів ГОСТ 28147-89 та RIJNDAEL?
- 2.4.12 У чому полягає режим простої заміни?
- 2.4.13 У чому полягає режим гамування?
- 2.4.14 У чому полягає гамування зі зворотним зв'язком?
- 2.4.15 У чому полягає режим виробітки імітовставки?

3 ЛАБОРАТОРНА РОБОТА №3

СТЕГАНОГРАФІЧНИЙ ЗАХИСТ

Мета роботи: ознайомитися з методами стеганографічного захисту інформації.

3.1 Короткі теоретичні відомості

У ряді систем необхідно забезпечити не тільки конфіденційність, цілісність і дійсність переданих команд, але й сховати сам факт передачі цієї інформації. Цим займається стеганографія.

Під стеганографією розуміється спосіб тайнопису, який більш давніший ніж сама криптографія. Основною метою криптографії є забезпечення конфіденційності, цілісності, спостережності та доступності за рахунок використання методів криптографічного захисту інформації.

Стеганографія – це наука про приховання однієї інформації серед іншої. Це метод організації зв'язку, що власне ховає саму наявність зв'язку.

На відміну від криптографії, де ворог точно може визначити чи є передане повідомлення зашифрованим текстом, методи стеганографії дозволяють вбудовувати секретні повідомлення в загальні послання так, щоб неможливо було запідозрити існування убудованого таємного послання.

Термін «стеганографія» у перекладі з грецького буквально означає «тайнопис» (steganos – секрет, таємниця; graphy – запис). До неї відноситься величезна множина секретних засобів зв'язку, таких як невидиме чорнило, мікрофотознімки, умовне розташування знаків, таємні канали і засоби зв'язку на частотах, що плавають та інше.

Стеганографія займає свою нішу в забезпеченні безпеки: вона не заміняє, а доповнює криптографію. Приховування повідомлення методами стеганографії значно знижує ймовірність виявлення самого факту передачі повідомлення. А якщо це повідомлення до того ж зашифроване, то воно має ще один, додатковий, рівень захисту.

В даний час у зв'язку з бурхливим розвитком обчислювальної техніки і нових каналів передачі інформації з'явилися нові стеганографічні методи, в основі яких лежать особливості

представлення інформації в комп'ютерних файлах, обчислювальних мережах і т.д. Це дає можливість говорити про становлення нового напрямку – комп'ютерної стеганографії.

3.1.1 Стеганографічна система

Стеганографічна система чи стегосистема (рис. 3.1) – це сукупність засобів і методів, що використовуються для формування схованого каналу передачі інформації.

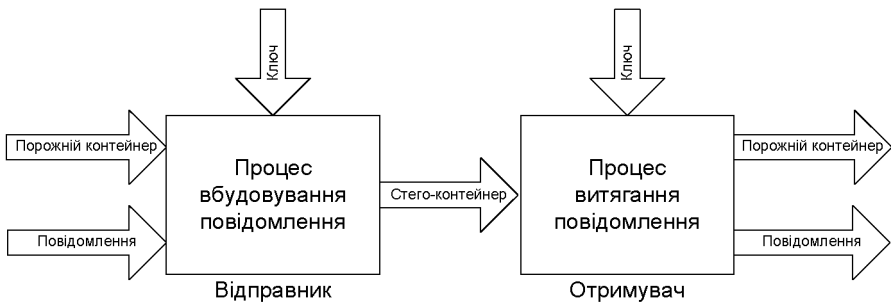


Рисунок 3.1 – Стеганографічна система

При побудові стегосистеми повинні враховуватися наступні положення:

а) супротивник має повне представлення про стеганографічну систему і деталі її реалізації. Єдиною інформацією, що залишається невідомою потенційному супротивнику, є ключ, за допомогою якого тільки його власник може установити факт присутності і зміст схованого повідомлення;

б) якщо супротивник якимсь образом довідається про факт існування схованого повідомлення, це не повинно дозволити йому витягти подібні повідомлення в інших даних доти, поки ключ зберігається в таємниці;

в) потенційний супротивник повинний бути позбавлений будь-яких технічних та інших переваг у розпізнаванні чи розкритті змісту таємних повідомлень.

Контейнер – будь-яка інформація, призначена для приховання таємних повідомлень.

Порожній контейнер – контейнер без вбудованого повідомлення; заповнений контейнер чи стего-контейнер, що містить вбудовану інформацію.

Вбудоване (сховане) повідомлення – повідомлення, що вбудовується в контейнер.

Стеганографічний канал чи просто стегоканал – канал передачі стего-контейнера.

Стегоключ чи просто ключ – секретний ключ, необхідний для приховування інформації. У залежності від кількості рівнів захисту (наприклад, вбудовування попередньо зашифрованого повідомлення) у стегосистемі може бути один чи декілька стегоключів комп'ютерної стеганографії.

3.1.2 LSB-метод (Least Significant Bits) приховування інформації

В даний час найбільш розповсюдженим є метод заміни найменших значущих бітів чи LSB-метод. Він полягає у використанні похибки дискретизації, що завжди існує в діджиталізованих зображеннях чи аудио- або відеофайлах. Дана похибка дорівнює найменшому значущому розряду числа, що визначає величину колірної складової елемента зображення (пікселя). Тому модифікація молодших бітів у більшості випадків не викликає значної трансформації зображення та не виявляється візуально.

Суть методу така. Цифрові картини складаються з окремих крапок, так званих пікселів. Якщо зображення 8-бітове, це значить, що для опису одного пікселя картини приділяється один байт графічного файлу. Один байт, як відомо, складається з восьми біт, кожний з яких може дорівнювати нулю чи одиниці. Програма – стеганограф теж повинна записати файл-повідомлення нулями й одиницями, і вона розміщає ці біти поверх самих останніх восьми бітів кожного байту в графічному файлі. Якщо нуль чи одиниця файлу-повідомлення волею случая збігаються з нулем чи одиницею файлу-контейнера, то ніяких змін не відбувається. Якщо ж вони не збігаються, програма – стеганограф записує останній біт так, як їй потрібно.

Звичайно, після цього весь байт графічного файлу змінює своє значення, але оскільки змінюється тільки останній його біт – найменш значущий, то байт змінюється тільки на одиницю. Наприклад, якщо спочатку байт дорівнює п'ятдесятьом, то після стеганографії він дорівнює п'ятдесят одному чи сорока дев'яти. Така зміна, звичайно, змінює колір одного пікселю, що відноситься до зміненого байту. Але тому що байт змінюється тільки на одиницю, то і відтінок пікселю змінюється незначно та помітити його на око практично неможливо, особливо якщо мова йде про чорно-білу фотографію, оскільки градації сірого людина розрізняє набагато гірше кольорових відтінків.

В додавок до цього, графічний файл може бути не 8-ми, а 24-бітним, відомий true-color, у якому кожен піксель може мати більш 16 мільйонів відтінків. Крім того, якщо повідомлення, що ховає стеганограф, маленьке, то зовсім не обов'язково змінювати кожен байт картини – цілком достатньо може виявитися кожного десятого, чи навіть кожного сотого байту стеганографії.

3.1.3 Формат BMP-файлу

Файли формату BMP (скорочено від BitMaP – бітовий образ) зберігають зображення в True Color. Розглянемо формат файлу BMP для 24-розрядного рисунку. Файл BMP містить заголовок – 54 байти та бітовий образ зображення. Кожна точка зображення – піксель (picture element) – описується трьома байтами, що включають складові частини кольору – червона (Red), зелена (Green), синя (Blue) (рис. 3.2). Інтенсивність складових частин кольору змінюється в межах від 0 до 255. Шляхом варіації інтенсивності кожної складової частини можна змінювати колір від чорного, коли інтенсивність всіх складових частин дорівнює 0 (00 00 00), до білого, коли інтенсивність всіх складових частин максимальна і дорівнює 255 (FF FF FF). Наприклад, точка червоного кольору задається як (255,0,0) або (FF 00 00).

Зображення записується в файл по рядкам. Першим сканується нижній рядок (зліва направо). Скан – рядки вирівняні по 32-бітній границі (4 байти). Тобто, якщо ширина зображення не кратна 4, то інформація про строку доповнюється нульовими байтами.



Рисунок 3.2 – Представлення кольору трьома складовими частинами RGB

Розмір заголовку 24-розрядного рисунку – 54 байти. У табл. 3.1 наведено призначення окремих байтів заголовку BMP – файлу.

Таблиця 3.1 – Призначення байтів заголовку BMP – файлу

1-2 байти:	тип файлу – BM (bit mapping);
3-6 байти:	розмір файлу;
7-10 байти:	не використовуються;
11-14 байти:	зсув даних бітового образу від заголовку в байтах – 54;
15-18 байти:	число байт до початку бітового образу – 40;
19-22 байти:	ширина бітового образу в пікселях;
23-26 байти:	висота бітового образу в пікселях;
27-28 байти:	число бітових площин пристрою – 1;
29-30 байти:	число бітів на піксель – 24;
31-34 байти:	тип стискання – 0 (без стискання);
35-38 байти:	розмір картини у байтах;
39-42 байти:	горизонтальна здатність пристрою, піксель/м;
43-46 байти:	вертикальна здатність пристрою, піксель/м;
47-50 байти:	кількість кольорів, що використовуються – 0 (всі кольори);
51-54 байти:	кількість "важливих" кольорів.

Далі йдуть дані бітового образу картинки. Кожний піксел представляється трьома байтами – інтенсивностями червоного, зеленого, синього.

3.2 Завдання на лабораторну роботу

Реалізувати метод LSB у пакеті Visual C++. В якості контейнера застосувати файл формату *.bmp. В якості повідомлення використати своє «Прізвище Ім'я По-батькові».

3.3 Зміст звіту

- 3.3.1 Титульний лист, тема і мета роботи.
- 3.3.2 Відповіді на контрольні питання.
- 3.3.3 Текст програми.
- 3.3.4 Результати роботи програми.
- 3.3.5 Висновки.

3.4 Контрольні питання

- 3.4.1 Чим відрізняється стеганографія від криптографії?
- 3.4.2 Що таке стегоконтейнер?
- 3.4.3 В чому сутність метода LSB?
- 3.4.4 Як використовуються цифрові водяні знаки?
- 3.4.5 Які недоліки стеганографічного захисту?
- 3.4.6 Що таке стеганографічний метод захисту інформації?
- 3.4.7 Що таке контейнер?
- 3.4.8 Що може виступати як контейнер?
- 3.4.9 Вимоги, пропоновані до стего-повідомлень?
- 3.4.10 Що таке цифрові водяні знаки?
- 3.4.11 Які методи вбудовування повідомлень ви знаєте?
- 3.4.12 Що в даній програмі може бути використано як ключ?

4 ЛАБОРАТОРНА РОБОТА №4

ДОСЛІДЖЕННЯ РОБОТИ ПРОГРАМ-МОНІТОРІВ

Мета роботи: ознайомитись на практиці з основними програмними засобами, що використовуються зламниками для аналізу роботи захищеного програмного забезпечення. Навчитись обирати програмні засоби для аналізу системи захисту програм.

Використовуване програмне забезпечення: програмні монітори - FileMon, Regmon, API Monitor, ProcessExplorer, PortMon, програми Process Monitor.

4.1 Теоретичні відомості

Для ефективного дослідження (зламу) програмного забезпечення (ПЗ) хакерам, перш за все, необхідно зібрати всю необхідну і вичерпну інформацію про роботу цього програмного засобу: про середовище, в якому воно виконується, про глобальні системні змінні та параметри, які він використовує або активно модифікує, про файли і папки, які він використовує для захисту. Крім того, необхідно визначити, чи здійснює дане ПЗ програмні прив'язки та які динамічні бібліотеки використовує.

Наближений алгоритм несанкціонованого дослідження, що його використовує зловмисник для зняття захисту з програми:

а) перш за все, необхідно запустити програмний засіб, спостерігати за його роботою. Це можуть бути підозрілі рядки, масиви символів, діалогові вікна для введення реєстраційної інформації тощо;

б) далі необхідно відслідкувати звернення програми до системного реєстру та власних файлів налаштувань, оскільки інколи можна лише змінити або видалити певні записи з реєстру або деякий файл для зняття всіх обмежень у роботі програми;

в) далі слід відслідкувати звернення програми до файлів, каталогів – до ресурсів файлової системи;

г) проаналізувати виконувані файли та динамічні бібліотеки на предмет запакування виконуваних файлів, їх шифрування тощо;

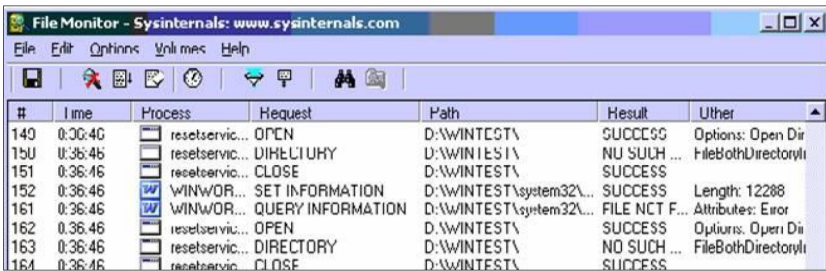
д) далі можна приступити до дизасемблювання і модифікації програмних модулів, тобто безпосередньо до здійснення зламу.

File Monitors – програми-монітори файлової системи, даний тип програмних продуктів дозволяє відслідковувати зміни, що

відбуваються у файловій системі під час запуску певних програм. У більшості таких програм передбачена система фільтрів для формування протоколів роботи окремих додатків. За допомогою даного типу засобів реалізується аналіз роботи систем захисту програмного забезпечення (СЗПЗ) з файлами.

Подібні програми дозволяють з'ясувати, що саме і де змінюють розпізнані на етапах первинного і вторинного аналізу модулі СЗПЗ, або визначити модуль, що робить зміни у певному файлі. Ця інформація дозволяє точно локалізувати лічильники кількості запусків ПЗ, приховані файли систем "прив'язки" ПЗ, "ключові файли".

Важливою є інформація про звернення захищеної програми до файлів, оскільки навіть під час свого виконання програма зчитує команди з власного бінарного файлу до оперативної пам'яті, вже не кажучи про інші приховані файли (файли налаштувань, файли ключів). Для вирішення таких задач дослідникам допомагає програма FileMon (рис. 4.1).



The screenshot shows the File Monitor application window with a menu bar (File, Edit, Options, Utilities, Help) and a toolbar. Below is a table with the following columns: #, Time, Process, Request, Path, Result, and Other. The table contains several rows of system events, including file operations like OPEN, CLOSE, SET INFORMATION, and QUERY INFORMATION.

#	Time	Process	Request	Path	Result	Other
149	0:36:40	reset.servic...	OPEN	D:\WINTEST\	SUCCESS	Options: Open Dir
150	0:36:46	reset.servic...	DELETE	D:\WINTEST\	NO SUCH...	FileBothDirectoryl
151	0:36:46	reset.servic...	CLOSE	D:\WINTEST\	SUCCESS	
152	0:36:46	WINWOR...	SET INFORMATION	D:\WINTEST\system32\...	SUCCESS	Length: 12288
161	0:36:46	WINWOR...	QUERY INFORMATION	D:\WINTEST\system32\...	FILE NOT F...	Attributes: Error
162	0:36:46	reset.servic...	OPEN	D:\WINTEST\	SUCCESS	Options: Open Dir
163	0:36:46	reset.servic...	DIRECTORY	D:\WINTEST\	NO SUCH...	FileBothDirectoryl
164	0:36:46	reset.servic...	CLOSE	D:\WINTEST\	SUCCESS	

Рисунок 4.1 – Загальний вигляд вікна програми FileMon

У головному вікні програми у таблиці поля містять інформацію про стан об'єкту, про результати виконання операцій з файлами, час виконання певної дії.

Крім того, повідомлення можна відфільтровувати за певними параметрами та критеріями.

Registry Monitors – програми-монітори системних файлів ОС, програмні засоби цього типу призначені для відстеження змін, внесених додатками в конфігураційні файли операційних систем (ОС). У розглянутому контексті дані програми дозволяють реалізувати аналіз СЗПЗ із системними файлами (більш специфічно для ОС сімейства Windows).

Розглянуті засоби дозволяють визначати, чи працює система захисту з файлами конфігурації ОС, які зміни вона туди вносить і які дані використовує. У результаті подібного аналізу стає можливим знайти приховані лічильники кількості запусків ПЗ, збережені дати першої установки ПЗ на ЕОМ користувача, записи з ліцензійними обмеженнями функціональності ПЗ і т.д. Такий аналіз дає результати, подібні до результатів аналізу роботи СЗПЗ з файлами.

Прикладом такої програми може слугувати програма Regmon. Ця програма перехоплює звернення будь-якої іншої програми до системного реєстру, виводячи при цьому повну та докладну інформацію про ключі, до яких здійснювалось звернення (рис. 4.2).

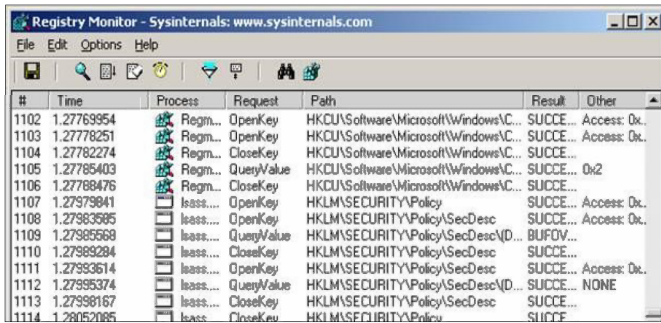


Рисунок 4.2 – Загальний вигляд вікна програми RegMon

Програма також дозволяє фільтрувати повідомлення за певним конкретним процесом.

Знаючи про звернення програми до системного реєстру можна встановити саме той режим роботи додатку, який необхідний зламнику, або взагалі позбутись всіх обмежень.

API Monitors – програми-монітори викликів підпрограм ОС, програмне забезпечення цього типу призначено для відстеження виклику системних функцій одним чи декількома додатками з можливістю фільтрації або видалення певних системних функцій або додатків. Застосування таких програм дозволяє проводити аналіз використання в СЗПЗ системних функцій.

З огляду на те, що всі дії ПЗ і СЗПЗ, які пов'язані з роботою з файловою системою, роботою з конфігурацією ОС, реалізацією діалогу з користувачем, роботою з мережею та іншим, реалізуються за

допомогою викликів функцій ОС. Аналіз використання СЗПЗ системних функцій дозволяє досить докладно вивчити механізми роботи систем захисту, знайти в них слабкі місця і розробити шляхи обходу впровадженого захисту.

Наприклад, практично всі сучасні системи захисту від копіювання оптичних дисків базуються на досить невеликому наборі системних функцій для роботи з даним видом накопичувачів інформації, і відстежування цих функцій дозволяє знайти та нейтралізувати механізми перевірки типу носія усередині СЗПЗ (рис. 4.3).

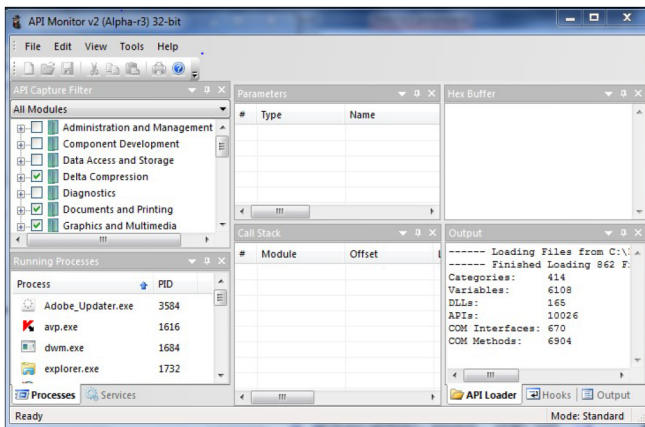


Рисунок 4.3 – Загальний вигляд вікна програми API Monitor

Окрім того, що зазначена програма дає можливість подивитись перелік API-функцій у використовуваній програмі. Вона дозволяє подивитись конкретні значення параметрів цих функцій при вході у систему захисту, а отже, отримати значення паролів, логінів, серійних номерів тощо.

Process/Windows Managers – програми-монітори активних задач, процесів, потоків і вікон, зазначений тип програмних засобів призначений для відстеження і керування об'єктами ОС (задачами, процесами, потоками, вікнами й ін.).

Подібні програми звичайно надають можливості пошуку необхідного об'єкта ОС, переключення на нього керування, зміни його пріоритету, знищення об'єкта, збереження його параметрів (іноді

вмісту) на диску. Застосування моніторів задач дає можливість здійснювати аналіз модульної структури СЗПЗ.

До таких засобів відносять програму Process Explorer (рис. 4.4).

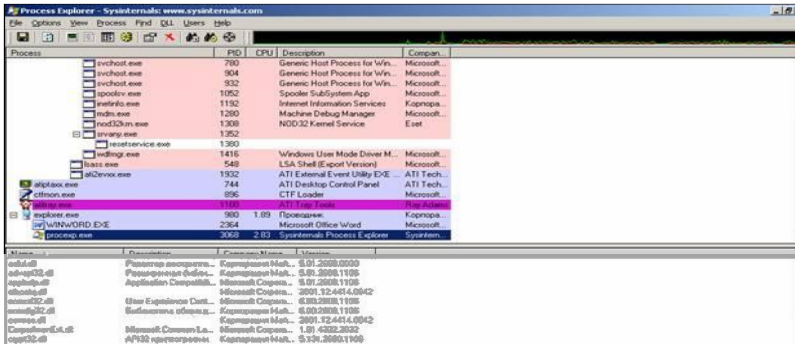


Рисунок 4.4 – Загальний вигляд вікна програми ProcessExplorer

Port Monitors – програми-монітори обміну даними із системними пристроями (портами).

У сучасній архітектурі ОС доступ до всіх системних пристроїв (іх контролерів) здійснюється через порти введення/виведення. Всі сучасні ОС віртуалізують ці порти, організовуючи в такий спосіб спільний доступ декількох додатків до одного й того самого порту (використовуючи механізм черг), а також здійснюючи контроль доступу до портів з метою забезпечення безпеки ОС.

Використання цього типу програмних засобів дозволяє проводити аналіз взаємодії СЗПЗ із системними пристроями.

Контролюючи доступ і обмін даними через порти введення/виведення програмної і апаратної частин СЗПЗ, можна аналізувати і переборювати механізми таких типів захистів, як СЗПЗ з електронними ключами, СЗПЗ з ключовими дисками і СЗПЗ "прив'язки" до архітектури комп'ютера користувача.

Прикладом програми для обміну даними із системними пристроями (портами) є програма PortMon (рис. 4.5).

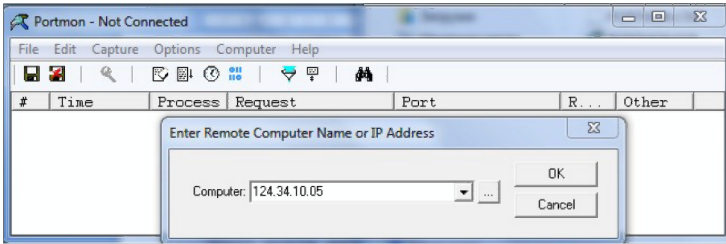


Рисунок 4.5 – Загальний вигляд вікна програми PortMon

Process Monitor – відстеження активності процесів, програма Process Monitor є вдосконалим інструментом стеження для Windows, який в режимі реального часу відображає активність файлової системи, реєстру, а також процесів і потоків (рис. 4.6).

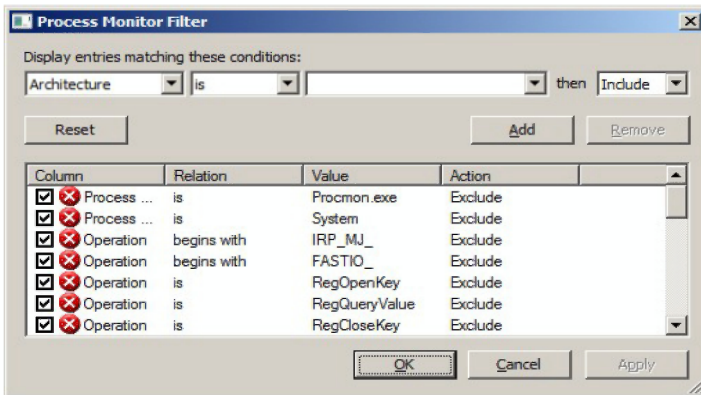
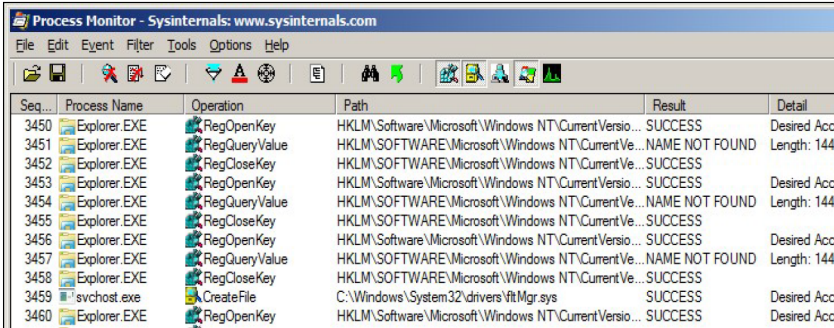


Рисунок 4.6 – Головне вікно програми Process Monitor

У цій програмі поєднуються можливості двох раніше випущених програм від Sysinternals: Filemon і Regmon, а також величезний ряд поліпшень, включаючи розширену та нешкідливу фільтрацію, всеосяжні властивості подій, такі як ID сесій і імена користувачів, достовірну інформацію про процеси, повноцінний стек потоку з вбудованою підтримкою всіх операцій, одночасний запис інформації у файл і багато інших можливостей.

Ці можливості роблять програму Process Monitor ключовим інструментом для усунення неполадок та позбавлення від шкідливих програм.

Інтерфейс користувача програми Process Monitor і параметри схожі з інтерфейсом і параметрами програм Filemon і Regmon, але в програмі Process Monitor є ряд істотних поліпшень (рис. 4.7):



The screenshot shows the Process Monitor window with the title bar 'Process Monitor - Sysinternals: www.sysinternals.com'. The menu bar includes 'File', 'Edit', 'Event', 'Filter', 'Tools', 'Options', and 'Help'. The toolbar contains various icons for file operations, search, and system functions. The main window displays a table of events with the following columns: Seq..., Process Name, Operation, Path, Result, and Detail.

Seq...	Process Name	Operation	Path	Result	Detail
3450	Explorer.EXE	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersio...	SUCCESS	Desired Acc
3451	Explorer.EXE	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	NAME NOT FOUND	Length: 144
3452	Explorer.EXE	RegCloseKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	SUCCESS	
3453	Explorer.EXE	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersio...	SUCCESS	Desired Acc
3454	Explorer.EXE	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	NAME NOT FOUND	Length: 144
3455	Explorer.EXE	RegCloseKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	SUCCESS	
3456	Explorer.EXE	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersio...	SUCCESS	Desired Acc
3457	Explorer.EXE	RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	NAME NOT FOUND	Length: 144
3458	Explorer.EXE	RegCloseKey	HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVe...	SUCCESS	
3459	svchost.exe	CreateFile	C:\Windows\System32\drivers\ftmMgr.sys	SUCCESS	Desired Acc
3460	Explorer.EXE	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersio...	SUCCESS	Desired Acc

Рисунок 4.7 – Фрагменти інтерфейсу програми Process Monitor

- а) відстежування запуску та завершення роботи процесів і потоків, включаючи інформацію про код завершення;
- б) відстежування завантаження образів (бібліотек DLL і драйверів пристроїв, що працюють в режимі ядра);
- в) збір стеків потоків для кожної операції дозволяє в більшості випадків визначити вихідну причину виконання операції;
- г) достовірний збір інформації про процеси, включаючи шлях до образу процесу, командний рядок, а також ID користувача і сесії.
- д) запис в журнал всіх операцій під час завантаження системи і інші.

4.2 Завдання на лабораторну роботу

Навчитись використовувати базові функції запропонованих програм, і заповнити таблицю характеристик.

4.3 Зміст звіту

- 4.3.1 Титульний лист, тема і мета роботи.
- 4.3.2 Відповіді на контрольні питання.
- 4.3.3 Короткі відомості.
- 4.3.4 Таблиця с результатами.
- 4.3.5 Висновки.

4.4 Контрольні питання

4.4.1 Навести загальний порядок здійснення зламу захисту.

4.4.2 Охарактеризувати програми-монітори звернень до файлів. Їх призначення.

4.4.3 Дати характеристику програмам стеження за системним реєстром. Їх використання для аналізу систем захисту.

4.4.4 Дати характеристику програмам для моніторингу процесів і вікон.

4.4.5 Охарактеризувати програми-монітори API-викликів. Де і як їх можна застосувати?

4.4.6 Охарактеризувати особливості використання програм сканування портів, програм моніторингу мережевого обміну.

4.4.7 Для чого можуть бути використані вказані програми при покращенні роботи операційних систем?

4.4.8 Як можуть бути використані програми моніторингу при дослідженні роботи систем захисту програмного забезпечення?

4.5 Порядок виконання роботи

4.5.1 Ознайомитись з теоретичними відомостями про засоби моніторингу, наведеними в лекціях та в описі даної лабораторної роботи.

4.5.2 Ознайомитись з коротким описом програм-моніторів, встановити та запустити на виконання програми, що здійснюють моніторинг різних об'єктів операційної системи (вони знаходяться у папці Programs):

- моніторинг звернень до системного реєстру – RegMon;
- моніторинг звернень до дискової системи комп'ютера – DiskMon;
- моніторинг звернень до файлів та каталогів – FileMon;
- моніторинг звернень до портів – PortMon;
- перегляд інформації про використовувані API-функції – APIMon;
- моніторинг мережного обміну – Network Traffic Monitors;
- моніторинг активності операційної системи – Process Explorer, Process Monitor.

4.5.3 За допомогою програм-моніторів дослідити роботу таких програм:

- програми, що містяться у папці Examples;
- програми, розроблені в результаті виконання курсові роботи з ТП.

4.5.4 Проаналізувати отримані результати та оформити звіт з лабораторної роботи за такою формою (бажано у вигляді таблиці за наведеною далі формою):

- призначення програми, яка є об'єктом дослідження;
- відомості, які надає кожна з програм моніторингу при слідкуванні за програмою-об'єктом;
- оцінити кожную програму (за 5-бальною шкалою) відповідно до її функціональності, зручності у використанні, частоті оновлень тощо;
- зробити висновки щодо ефективності використаного у програмі об'єкти захисту.

Нижче наведено заголовки таблиці.

№	Назва програми	Призначення	Об'єкт дослідження	Результати дослідження	Оцінка	Висновки (позитивні і негативні)
1	2	3	4	5	6	7
1	RegMon	Моніторинг звернень до системного реєстру	Programm1	Програма звертається лише до службової інформації і не містить ключової інформації у реєстрі	3	Позитивні: зручний інтерфейс, наявність фільтрів. Недоліки: забагато зайвої інформації, ...

5 ЛАБОРАТОРНА РОБОТА №5

ЗАХИСТ ПРОГРАМ ЗА ДОПОМОГОЮ ПРОГРАМ- ПАКУВАЛЬНИКІВ

Мета роботи: Дослідити роботу основних програм для захисту виконуваних файлів шляхом пакування. Ознайомитись на практиці з програмами для ідентифікації пакування. Дослідити програми – розпакувальники, використовувані зламниками для зняття захистів.

Використовуване програмне забезпечення: UPX, UPX X-Shell, ASPack, PE Compact, HidePE, PEId.

5.1 Теоретичні відомості

Для захисту програмного забезпечення від несанкціонованого дослідження і використання розробники використовують пакувальники. Пакувальники, як правило, зменшують обсяг виконуваних програм, залишаючи при цьому їх повністю функціональними. А от одним із основних кроків, що їх використовують зламники захистів програмного забезпечення, є використання розпакувальників.

Проаналізувавши виконувані файли та динамічні бібліотеки на предмет запакування та їх шифрування, зламники можуть застосувати певні програмні засоби для зняття захистів. Приклади таких програм – популярні в Інтернеті WinZip, WinRAR та 7Zip.

Спростивши справу можна представити так. Використовуючи спеціальні алгоритмічні методи, архіватор знаходить у файлах часто повторюючі послідовності і заміняє їх більш короткими кодами. Наприклад, в текстових файлах часто повторюються деякі літери, "е", "а" або знак проміжку. Архіватор обчислює число входжень символів у тексті, а потім будує оптимізовану таблицю символів, в якій найбільш використовуючі символи мають самі короткі коди, як в азбуці Морзе.

Весь текст перекодується відповідно новій таблиці, і процес повторюється спочатку. Адже часто зустрічаються не лише окремі літери, але і їх послідовності, наприклад сполучення "пр", "ст" або " ". Перекодування файлу завершується, коли його розмір після оптимізації перестає зменшуватись.

Зазвичай, архіватори дозволяють ущільнювати не лише текстові файли. Оскільки текст представлений у вигляді набору кодів, архіватор з тим же успіхом ущільнює і нетекстові файли.

Наприклад, у файлах програм типу .EXE або .DLL деякі коди команд зустрічаються частіше. Окрім цього, багато компонувачів здійснюють вирівнювання, при якому окремі сегменти програми доповнюються нулями до тих пір, поки розмір сегмента не стане кратним визначеній величині, наприклад, 8 або 16 байтам. Відповідно, програмні файли містять багато "води", яку можна "вижати" архіватором. В середньому, файли програм ущільнюються приблизно на 40-50%.

Ущільнення досить вигідне для поширення інформації, оскільки дозволяє без втрат розмістити її на носії меншого розміру і значно скоротити час завантаження по мережі. Але архіви створені звичайними програмами ущільнення, мають один вагомий недолік - файли в них недоступні безпосередньо. Для подальшого використання файли потрібно розпакувати із архіву. Як правило, це виконується тією самою програмою, що використовувалась для запакування. Також існують і саморозпаковуючі архіви, але суті справи це не змінює. Просто для розпакування такого архіву жодних додаткових програм не вимагається. А файли з нього для прямого використання все одно недоступні.

Але існує багато великих програм, якими ми користуємось рідко, але утримувати їх в архіві незручно. Зручніше запускати програму, не розпаковуючи. Давно створений цілий клас програм спеціально для цієї мети. Вони дозволяють упаковувати програму, отримуючи виконуючий файл меншого розміру. Робота пакувальника багато в чому схожа на роботу архіватора. По суті, пакувальник є спеціалізованим архіватором, що створює виконуючі файли програм. Результат його роботи – саморозпаковуваний архів, що містить програму. При запуску програма сама себе розпаковує, а потім починає працювати.

Дії пакувальника такі. Основний код програми (EXE, DLL та ін.) посегментно запаковують тим або іншим методом (LZW, ZIP и пр.). Потім в початок програми додається процедура її розпакування перед виконанням, і модифікований файл записується на диск. Пакувальники програм для Windows ще витягують із файлу ресурсів

основну іконку додатка, щоб файл програми в "Провіднику" виглядав достойно.

Варіантів пакування і розпакування досить багато. Самий простий варіант: програма при запуску розпаковуються повністю. В деяких випадках, спрямованих на боротьбу проти хакерів, частини програми розпаковуються лише по мірі звернення до них (понижаючи швидкодію). Більшість пакувальників працюють по методу "із пам'яті в пам'ять", тобто програма розпаковується в новий сегмент пам'яті і потім з нього запускається.

Отже, позитивні риси пакувальників такі:

- упакована програма залишається повністю функціональною;
- пакування програми є найбільш простим методом захисту її від зламу непрофесійним хакером-крекером;
- крім того, ущільнена програма певним чином захищена від вірусів, оскільки її основний код недоступний для модифікацій вірусом.

Однак, існує ряд негативних моментів пакувальників:

- пакування програми збільшує час завантаження: перш ніж починати роботу, програма повинна бути розпакована;
- процедура розпакування програми займає пам'ять. Адже при запуску пам'ять виділяється не програмі, а її розпакувальнику;
- після розпакування програма робить запит на нову пам'ять для своєї роботи, розмір якої завжди буде більший, ніж початково виділений даному процесу;
- перерозподіл пам'яті і копіювання розпакованої програми може значно знизити швидкодію системи. Це досить актуально для Windows всі програми, які виконуються у віртуальному адресному просторі, і деякі частини програми можуть бути вивантажені диспетчером пам'яті на диск. А це призводить до зростання числа звернень до диску і збільшення розміру файлу підкачки.

Отже, пакування програми не так вигідне, як здається з першого погляду. Тим не менше, пакування файлів все таки ускладнює несанкціоноване дослідження програм дизасемблерами.

5.1.1 UPX Packer

UPX Packer – це поширений пакувальник виконуваних файлів форматів COM, EXE та DLL-бібліотек, який на 50-70% зменшує розмір файлу та час його завантаження. Завдяки цьому програму практично неможливо дизасемблювати, оскільки вона повинна розпаковувати себе під час запуску. Програма має зручний інтерфейс, надає можливість задавати параметри пакування та створити резервні копії файлу (рис. 5.1).

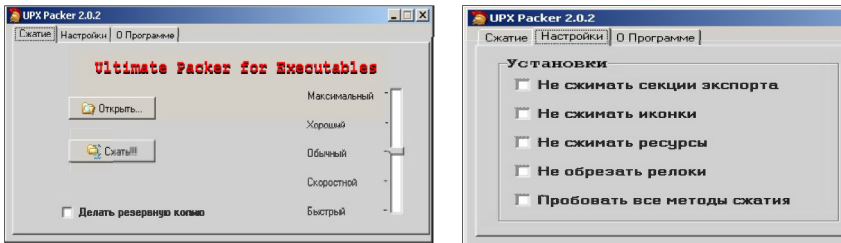


Рисунок 5.1 – Вигляд головного вікна програми UPX

Програма після пакування є абсолютно працездатною, лише під час запуску вона розпаковується, що займає дещо більше часу на її виконання, звичайно, при інших режимах пакування ступінь ущільнення є іншою (при цьому процес пакування проходить значно швидше), режим «Хороший» – 64,5%; «Звичайний» – 63%; «Швидкісний» – 57,5%; «Швидкий» – 56%.

5.1.2 UPX X-Shell

UPX X-Shell – зручний пакувальник-розпакувальник, призначений для пакування виконуваних файлів. Має дуже зручний інтерфейс (рис. 5.2).

Програма дозволяє здійснювати багато налаштувань, вибирати якість і швидкість пакування, використовувати різні методи пакування і т. д., від чого залежить і обсяг упакованого файлу.

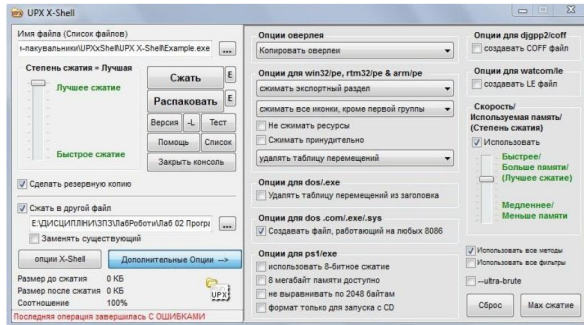


Рисунок 5.2 – Видяг голавного вiнка програми UPX X-Shell

5.1.3 ASPack

ASPack – удосконалена програма для ущiльнення виконуваних файлиб i iх захист вiд непрофесiйного аналізу та реверсивного iнженiрингу. Дана програма зменшує розмiри файлиб EXE i DLL-бiблiотек Windows до 40-70% (степiнь ущiльнення перевищує стандарт ZIP на 10-20%), а також скорочує час завантаження таких додаткiв у мережах iнтернет. Програми, запакованi ASPack, є автономними i запускаються так само, як i до пакування, без втрати часу i погiршення якостей. Особливостi програми:

- шифрування i ущiльнення програмного код, даних i ресурсiв (рис. 5.3);
- захист ресурсiв i коду вiд дизасемблерiв, декомпiляторiв i дамперiв;
- цiлком прозора, автономна робота з пiдтримкою довгих iмен файлиб;
- швидка процедура розпакування у порiвняннi з продуктами конкурентiв;
- безпосередня iнтеграцiя в оболонку Windows, що спрощує схему роботи;
- сумiснiсть з виконуваними файлами, згенерованими Visual C++, Visual Basic, Inprise (Borland) Delphi, C++ Bilder та iн.

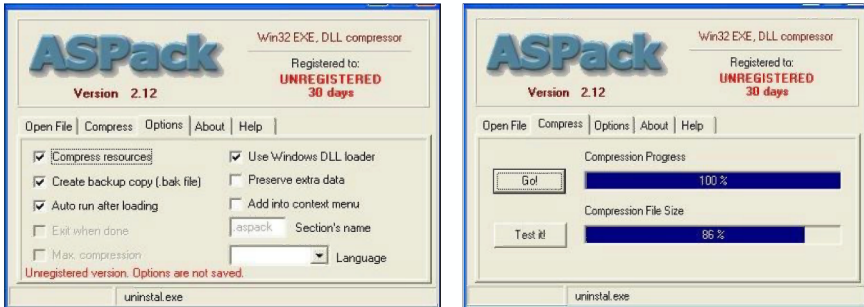


Рисунок 5.3 – Видгляд вікон пакувальника ASPack

Легко переконатись, що після пакування файлу упакована програма продовжує працювати. Для перевірки можна натиснути кнопку «Test It!» (або можна зайти в директорію з програмою і запустити її).

5.1.4 Hide PE

Даний пакувальник дещо відрізняється від інших. Сутність захисту полягає у тому, щоб, по-перше, виключити можливість автоматичного розпакування програм і по-друге, підмінити інформацію про реальний компілятор на іншу, тобто зламник буде вважати що файл захищено одним пакувальником, тоді як він запакований іншим (рис. 5.4).

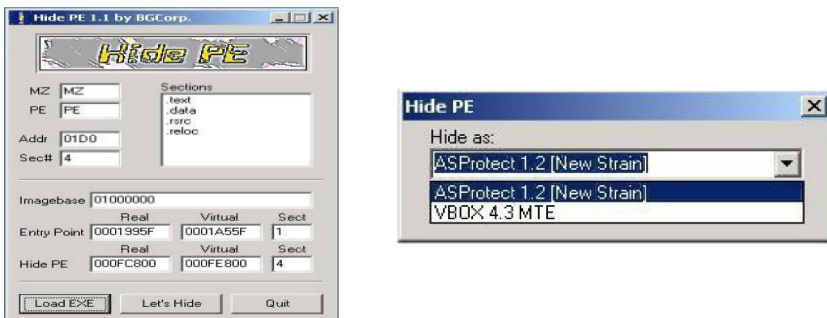


Рисунок 5.4– Видгляд вікон програми HidePE з завантаженням файлом

5.1.5 PE Compact

Дана програма призначена для захисту ПЗ від дослідження і використання і працює з *.EXE та *.DLL файлами (рис. 5.5). Для кожного файлу можна налаштувати власні опції пакування, можливість вибрати ресурси, що підлягають ущільненню, плагіни перехоплювачів API, рівень ущільнення, введення водяного знаку тощо.

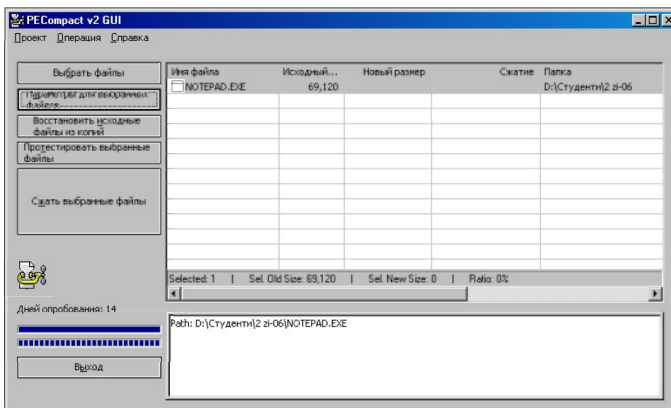


Рисунок 5.5 – Видгляд головного вікна програми PE Compact

Крім наведених, є цілий ряд різноманітних пакувальників, інформацію про які легко знайти у мережі.

Якщо існують програми-пакувальники, то повинні існувати і програми-розпакувальники. Адже, як зазначалось раніше, пакування програми захищає її від злому непрофесіоналом. Розпакувальників також є чимало.

Існує два принципово різних методи розпакування запакованих програм, метод орієнтований на конкретний пакувальник, розпакування методом трасування.

Розпакувальники першого типу інколи входять до складу пакувальників, наприклад, PKLite. Суть даного методу така – розпакувальник виконує дії з програмою, протилежні тим, що виконав пакувальник. Перевага такого підходу – отримання в результаті розпакування програми, повністю ідентичній тій, що була раніше. Проте, існує недолік – розпакувальник прив'язаний до конкретного

пакувальника або навіть до окремої версії. Якщо файл не розпізнаний як «власний», то розпакувальник його обробляти відмовляється.

Другий метод розпакування, – трасування, – принципово інший. Він був створений хакерами. Трасувальник запускає програму на виконання, а потім входить в режим відлагоджувача і намагається «засікти» той момент, коли розпаковка програми закінчиться, і їй буде передане керування.

Якщо це вдалось, трасувальник знаходить сегменти програми, виконує їх компоновку і записує на диск. Отриманий в результаті файл не буде байт в байт ідентичний вихідній програмі, точно як методи компоновки навряд відрізняються у трасувальника та вихідного компонування програми. З іншої сторони, для операційної системи порядок запису сегментів у файлі не має ніякого значення, якщо програма працює правильно. Таким чином, трасування є універсальним способом розпакування програм.

Правда, не завжди вдається «упіймати» момент запуску розпакованої програми. Існують методи маскування, направлені проти трасувальників. Один з них розглядався при огляді пакування програм – розпакування частин програми у міру звертання до них. Якщо під час виконання програма ніколи не розпаковується повністю, то жоден трасувальник не допоможе. Хоча, як і методи захисту, методи зламу постійно удосконалюються.

5.1.6 Ідентифікація параметрів пакування виконуваного файлу

Із зазначеного вище ми знаємо, що для захисту програм розробники використовують програми для пакування виконуваних файлів. Це стає “перешкодою” зловмиснику для здійснення зламу захищеної програми.

Для визначення, якою програмою-пакувальником упаковано захищений файл, можна скористатись програмою PEId. Дана програма також дозволяє аналізувати не один файл, а й всі файли в обраній папці. Також є можливість рекурсивного аналізу, що охоплює всі підпапки обраної нами.

Програма має вбудований менеджер задач, що дозволяє переглянути, які динамічні бібліотеки використовують програми в даний час (рис.5.6).

У мережі можна знайти багато цікавої інформації і про пакувальники, і про роз пакувальники, і про програми ідентифікації пакування, а також багато конкретних прикладів цих програм.

5.2 Завдання на лабораторну роботу

Навчитись використовувати базові функції запропонованих програм, і заповнити таблицю характеристик.

5.3 Зміст звіту

5.3.1 Титульний лист, тема і мета роботи.

5.3.2 Відповіді на контрольні питання.

5.3.3 Короткі відомості.

5.3.4 Таблиця с результатами.

5.3.5 Висновки.

5.4 Контрольні питання

5.4.1 В чому різниця між архіваторами та пакувальниками?

5.4.2 Який принцип дії програм-пакувальників?

5.4.3 Які позитивні і негативні риси пакувальників?

5.4.4 Назвіть декілька програм для пакування виконуваних файлів.

5.4.5 Наведіть приклади програм для ідентифікації пакування файлів. Яке їх призначення?

5.4.6 Яке програмне забезпечення виконує злам програмних продуктів, захищених пакувальниками?

5.4.7 Назвіть основні методи, що їх використовують розпакувальники.

5.5 Порядок виконання роботи

5.5.1 Ознайомитись з теоретичними відомостями про програми пакування, розпакування виконуваних файлів.

5.5.2 Підготувати для роботи декілька програм (результати виконання лабораторних і курсових робіт I-III курсів):

- виконуваний файл консольної програми малого обсягу;
- виконуваний файл консольної програми великого обсягу;
- виконуваний файл API-додатку під Windows;
- виконуваний файл MFC-додатку під Windows.

5.5.3 Підготувати не менше п'яти програм-пакувальників (бажано завантажити з Інтернету сучасні версії програм).

5.5.4 Підготувати дві програми для ідентифікації пакування програм.

5.5.5 Для кожної з програм-об'єктів виконати такі дії:

– запустити на виконання програму, попередньо зафіксувавши її обсяг;

– запакувати по черзі кожним з пакувальників, попередньо ознайомившись з їх можливостями і особливостями. При цьому зафіксувати обсяг і час виконання упакованих програм. Зберігати всі варіанти упакованих програм (для проведення подальших досліджень);

– дослідити захищену програму на предмет можливості розпакування програмами для ідентифікації пакування після кожного пакування;

– розпакувати упаковану програму, використовуючи відповідні розпакувальники, і дати характеристику якості розпакування. Порівняти отриманий в результаті розпакування файл з початковим.

5.5.6 Оформити звіт, в якому вказати (результати звести у табл. 5.1 за наведеною далі формою).

Таблиця 5.1– Результати дослідження

№	Назва програми та її обсяг	Пакувальник	Можливості пака	Результати пакування		Результати ідентифікації	Розпакувальник	Результат розпакування	
				Обсяг %	Зменшення %			Обсяг %	Працює (так-ні)
1	2	3	4	5		6	7	8	

5.5.7 Сформулювати висновки щодо методів та принципів роботи програм пакування і розпакування файлів та програм-ідентифікаторів пакування. Оцінити кожну з програм-пакувальників і розпакувальників за п'ятибальною шкалою, пояснити і обґрунтувати свою оцінку.

6 ЛАБОРАТОРНА РОБОТА №6

ДОСЛІДЖЕННЯ СТРУКТУРИ ВИКОНУВАНИХ ФАЙЛІВ

Мета роботи: Ознайомитись на практиці зі структурою заголовків виконуваних файлів. Дослідити структуру конкретних виконуваних файлів. Навчитись виділяти структурні елементи ехе-файлів, аналізувати їх з метою використання при захисті програмного забезпечення шляхом маніпулювання ехе-заголовками.

6.1 Стислі теоретичні відомості

В операційних системах, що входять у сімейство Win32, існує декілька типів виконуваних файлів. Ці типи розрізняються за розширеннями файлів, а також за сигнатурою. Сигнатури виконуваних файлів перераховані у заголовчному файлі <winnt.h> (табл. 6.1).

Таблиця 6.1 – Сигнатури виконуваних файлів

№	Сигнатура	Значення	Опис
1	Image dos Signature	0x5A4D	MZ
2	Image os2 Signature	0x454E	NE
3	Image os2 Signature le	0x454C	LE
4	Image VXD Signature	0x454C	LE
5	Image NT Signature	0x00004550	PE00

Ми розглянемо лише формат PE-файлів, оскільки саме з цими типами виконуваних файлів ми матимемо справу.

Чому саме DOS? Тому що виконуваний файл Windows є одночасно і виконуваним файлом DOS. Загальна схема виконуваного PE-файла наведена нижче (табл. 6.2).

Ехе-програми можуть складатися з декількох сегментів (кодів, даних, стека). Ехе-файл має заголовок, що використовується при його завантаженні. Заголовок складається з форматированої частини, що містить сигнатуру і дані, необхідні для завантаження Ехе-файла, і таблиці для настроювання адрес (Relocation Table). Таблиця складається зі значень у форматі. До зсувів у завантажувальному модулі, на яких указують значення в таблиці, після завантаження

програми в пам'ять повинна бути додана сегментна адреса, з якої завантажена програма.

Таблиця 6.2 – Загальна схема виконуваного PE-файла

№	Опис
1	old-exe-заголовок – заголовок EXE-файла DOS
2	PE File Signature – сигнатура PE-файла
3	PE-заголовок – заголовок виконуваного файлу Windows
4	Таблиця об'єктів (розділів)
5	Розділи

Заголовок розташовується на початку файлу EXE-програми. Опис його знаходиться у заголовочному файлі <winnt.h> і має таку структуру:

```
typedef struct _IMAGE_DOS_HEADER // DOS .EXE header
{
    WORD e_magic;           // сигнатура EXE-файла
                           // (4D5Ah - 'MZ')
    WORD e_cblp;            // довжина останньої сторінки
    WORD e_cp;              // довжина файлу в
                           // сторінках (по 512 б)
    WORD e_crlc;            // число елементів у таблиці
                           // настроювання адрес
    WORD e_cparhdr;         // довжина заголовка в параграфах
                           // (по 200h б)
    WORD e_minalloc;        // мінімальний і максимальний обсяг
                           // пам'яті,
    WORD e_maxalloc;        // яку потрібно виділити після
                           // завантаженого
    WORD e_ss;              // зсув сегмента стека
                           // (для установки SS)
    WORD e_sp;              // значення SP, установлене
                           // при вході
    WORD e_csum;            // контрольна сума
    WORD e_ip;              // значення IP при вході
                           // (стартова адреса)
    WORD e_cs;              // зсув сегмента коду
                           // (для установки CS)
    WORD e_lfarlc;          // зсув 1-го елементу таблиці
                           // налаштування адрес
```

```

WORD e_ovno;           // номер оверлею
                        // (0 - для кореневого модуля)
LONG e_lfanew;         // зміщення PE-заголовку
                        // від початку файлу
} IMAGE_DOS_HEADER, *PIMAGE_DOS_HEADER;

```

Якщо значення у слові, яке знаходиться зі зміщенням 18h, більше або рівне 40h, то за зміщенням 3Ch знаходиться значення зміщення, за яким знаходиться сигнатура якогось іншого виконуваного файлу. І якщо ця сигнатура представлена значенням «PE00», то файл є виконуваним файлом Windows. Отже, виконуваний файл Windows сам по собі існувати не може. Таким чином, визначили зміщення на початок виконуваного Windows файлу. А він також починається із заголовку.

У заголовочному файлі <winnt.h> цей заголовок описаний так:

```

typedef struct _IMAGE_ROM_HEADERS
{
    DWORD Signature;           // сигнатура виконуваного файлу
    IMAGE_FILE_HEADER FileHeader; // обов'язкова частина
    IMAGE_ROM_OPTIONAL_HEADER OptionalHeader;
                                // необов'язкова
};

```

Обов'язкова частина заголовку PE-файлу має таку структуру:

```

typedef struct _IMAGE_FILE_HEADER
{
    WORD Machine; // для якого процесора скомпільовано
// 0 - невідомий проц.; 0x14C - Intel 386,
// 0x14D - Intel 486, 0x14E - Intel Pentium і т.д.
    WORD NumberOfSections; // кількість розділів
    DWORD TimeDateStamp; // мітка часу створ. файлу
    DWORD PointerToSymbolTable; // зміщення таблиці симв.
    DWORD NumberOfSymbols; // кількість символів у
                            // таблиці символів
    WORD SizeOfOptionalHeader; // розмір необов'язкової
                                // частини WORD
    Characteristics; // 0x0100 - для 32-бітного
                    // комп'ютеру,
};
// 0x1000 - системний файл, 0x2000 - Dll-файл

```

Необов'язкова частина заголовка виконуваного файлу:

```

typedef struct _IMAGE_OPTIONAL_HEADER
{
    // Standard fields:
    WORD Magic; // 0x10b
    BYTE MajorLinkerVersion; // старші і молодші
                              // розряди
}

```

```

BYTE MinorLinkerVersion;      // номера версії лінкеру
DWORD SizeOfCode;             // сумарний розмір всіх
                               // розділів
DWORD SizeOfInitializedData   // розмір
                               // ініціалізованих даних
DWORD SizeOfUninitializedData; // неініціалізованих
                               // даних
DWORD AddressOfEntryPoint;    // відносна віртуальна
                               // адреса точки входу до
                               // програми(база - у
                               // змінній нижче)
DWORD BaseOfCode;             // відносна адреса
                               // програмної секції
DWORD BaseOfData;             // відносна адреса
                               // секції даних
                               // NT additional fields:
DWORD ImageBase;              // базова адреса файлу
                               // відображено у пам'яті
DWORD SectionAlignment;      // поля для вирівнювання
DWORD FileAlignment;
WORD MajorOperatingSystemVersion; // розряди номера
                                   // старої
WORD MinorOperatingSystemVersion; // версії, яка
                                   // дозволить
WORD MajorImageVersion;       // виконувати цей
                                   // файл
WORD MinorImageVersion;
WORD MajorSubsystemVersion;
WORD MinorSubsystemVersion;
DWORD Reserve1;               // резервні
DWORD SizeOfImage;            // розмір завантаж.
                               // модуля у пам'яті
DWORD SizeOfHeaders;          // розмір заголовків і
                               // таблиці розділів
DWORD CheckSum;               // контрольна сума
                               // виконуваного файлу
WORD Subsystem;               // тип підсистеми для
                               // інтерфейсу
WORD DllCharacteristics;      // 1 - виклик Dll при
                               // першому завантаженні
                               // до адресного простору
                               // процесу, 2 - інакше
DWORD SizeOfStackReserve;     // розмір потрібного
                               // стеку
DWORD SizeOfStackCommit;
DWORD SizeOfHeapReserve;
DWORD SizeOfHeapCommit;

```

```

DWORD LoaderFlags;           // не використовується
DWORD NumberOfRvaAndSizes;   // кількість елементів
IMAGE_DATA_DIRECTORY         // цього масиву

```

Таблиця об'єктів (або таблиця розділів, або таблиця секцій) – сукупність даних певного призначення: про експортовані та імпортовані функції, про ресурси, про переміщення (relocations) і т. д., які компактно розміщені у виконуваному файлі. Іншими словами, таблиця об'єктів – це просто інформація про зміст розділів.

Таблиця об'єктів розміщається відразу ж після заголовка PE-файла. у заголовочному файлі <winnt.h> існує опис структури розміром 40 байтів:

```

typedef struct _IMAGE_SECTION_HEADER
{
    // заголовок образу розділу
    BYTE Name[IMAGE_SIZEOF_SHORT_NAME];
    union {
        DWORD PhysicalAddress; // розмір розділу
        DWORD VirtualSize;
    } Misc;
    DWORD VirtualAddress;      // зміщення для цього розд.
    DWORD SizeOfRawData;      // округлений розмір розд.
    DWORD PointerToRawData;    // зміщення відносно
                                // початку файлу
    DWORD PointerToRelocations; // в ехе-файлах не викор.
    DWORD PointerToLinenumbers; // в ехе-файлах викор.
    WORD NumberOfRelocations;  // в ехе-файлах не викор.
    WORD NumberOfLinenumbers;  // практично не викор.
    DWORD Characteristics;     // атрибути розділу (для
                                // читання, для запису, чи
                                // кешується, чи містить
                                // ініційовані дані і т.п.
};

```

Розглянемо розділи, які зустрічаються частіше і те, яку інформацію вони містять.

.text, CODE – секція програмного коду. Ця секція містить код, який генерує компілятор. В компіляторах фірми Microsoft ця секція називається .text, а в компіляторах фірми Borland (нині Inprise) вона називається CODE. З цією секцією нерозривно пов'язані секції імпорту і експорту функцій.

.data, DATA – секція ініціалізованих даних. Назва цієї секції також залежить від виробника: .edata – для компіляторів Microsoft,

DATA – для Borland. У даній секції містяться глобальні та статичні змінні, літерали.

.bss – секція неініціалізованих даних. Оскільки ніяких даних у цій секції зберігати не потрібно, то зазвичай її розмір у виконуваному файлі дорівнює 0. Точніше, насправді такої секції у виконуваному файлі немає, а є лише згадування про неї у таблиці секцій. Компілятори фірми Borland цієї секції не створюють.

.idata (ImportDATA) – секція імпортованих функцій. У цій секції знаходяться таблиці, що забезпечують імпорт функцій з бібліотек динамічної компановки.

.edata (ExportDATA) – секція експортованих даних. Секція містить дані про ті функції, які експортуються даним модулем.

.rsrc (ReSouRCes) – секція ресурсів. Вона зберігає всі ресурси, які присутні у виконуваному файлі.

.reloc – секція містить таблицю базових поправок. Вона використовується лише тоді, коли завантажувальник може завантажити файл за адресою, починаючи з якої планував здійснити завантаження редактор зв'язків. .tls (Thread Local Section).

Якщо передбачається використовувати локальну пам'ять потоку, то ці дані не попадають в секції ініціалізованих або неініціалізованих даних. Замість цього вони попадають саме в дану секцію. У виконуваних файлах зустрічаються і інші секції.

6.2 Завдання на лабораторну роботу

Ознайомитись зі структурою заголовків виконуваних файлів. Виділити структурні елементи ехе-файлів, аналізувати їх з метою використання при захисті програмного забезпечення шляхом маніпулювання ехе-заголовками.

6.3 Зміст звіту

6.3.1 Титульний лист, тема і мета роботи.

6.3.2 Відповіді на контрольні питання.

6.3.3 Короткі відомості.

6.3.4 Таблиця с результатами.

6.3.5 Висновки.

6.4 Контрольні питання

6.4.1 Охарактеризуйте структуру виконуваних файлів.

6.4.2 Що таке заголовки виконуваних файлів, які їх види є?

6.4.3 Для чого операційна система використовує заголовки?

6.4.4 Що таке таблиця об'єктів (розділів виконуваного файлу)?

6.4.5 Які об'єкти (розділи) існують у програмах від різних виробників?

6.4.6 Які способи впровадження захисних механізмів у виконувані файли ви можете запропонувати?

6.5 Порядок виконання роботи

6.5.1 Вивчити і зрозуміти структуру виконуваного файлу.

6.5.2 Для виконання лабораторної роботи використати результати лабораторної роботи №2 (Пакувальники) і підготувати файли, які отримані в результаті пакування різними пакувальниками (використати не менше 3х різних пакувальників). Причому серед цих програм повинні бути програми, розроблені під MS-DOS та під Windows.

6.5.3 За допомогою програми Zagolovok:

– дослідити заголовки виконуваних модулів підготовлених програм;

– дослідити програмні засоби, використовувані для зчитування заголовків;

– прочитати і видати на екран таблицю розділів файлу.

6.5.4 За допомогою програм Hiew та PE Explorer проаналізувати структуру підготовлених виконуваних файлів. Результати дослідження можна оформити у вигляді табл. 6.3.

Таблиця 6.3 – Результати дослідження

№	Програма	Пакувальник	Заголовок		Розділи файлу	Спосіб упакування	Можливості використання для захисту
			Dos-заголовок	PE-заголовок			
1	2	3	4	5	6	7	8

7 ЛАБОРАТОРНА РОБОТА №7

РОЗРОБКА І РЕАЛІЗАЦІЯ ВБУДОВАНОГО ЗАХИСТУ ПРОГРАМ

Мета роботи: Ознайомитись з основними групами методів для захисту програм від дизасемблювання. Дослідити основні методи обфускації програмного коду. Знати види і рівні обфускації коду. Навчитись на практиці використовувати у програмах основні методи обфускації і заплутування коду. Вміти досліджувати коди розроблених програм до і після захисту з метою доведення його ефективності. Навчитись обґрунтовувати вибір програмних засобів для реалізації систем захисту програм.

7.1 Теоретичні відомості

Виділимо декілька найзагальніших підходів до захисту програмного забезпечення від дизасемблювання:

а) шифрування коду – один з найпоширеніших та надійних методів захисту від статичного дослідження;

б) маніпулювання заголовками EXE-файлів – метод, побудований на використанні властивостей структури виконуваних файлів і розробці навісного захисту;

в) обман дизасемблеру. Методи цієї групи полягають у тому, щоб заплутати дизасемблер, підсунути дані замість коду, дезорієнтувавши його логіку, повести його по помилковому сліду, підсунути зайві фрагменти коду і т.п. Всі ці способи обману можна поділити на такі групи:

- різноманітні методи обфускації коду;
- ускладнення логіки програм;
- різноманітні додаткові методи боротьби з інтерактивними та автоматичними дизасемблерами.

Це авангардні і досить перспективні методи захисту ПЗ не тільки від дизасемблювання, а й від налагоджування.

д) методи емуляції, серед яких виділяють:

- емуляцію процесора;
- емуляцію мультизадачності.

Обфускація ("obfuscation") – це один з методів захисту програмного засобу, який дозволяє ускладнити процес реверсивної

інженерії початкового коду програми, що захищається. Суть процесу обфускації полягає в тому, щоб заплутати програмний код і усунути більшість логічних зв'язків у ньому тобто трансформувати його так, щоб він був дуже важкий для вивчення і модифікації сторонніми особами (будь то зловмисники, або програмісти які збираються дізнатися унікальний алгоритм роботи програми, що захищається).

Обфускація може застосовуватися не тільки для захисту програмних продуктів, вона має ширше застосування (наприклад, може бути використана творцями вірусів для захисту їх творінь і т. д.).

Обфускація буває декількох видів:

- лексична обфускація (символьна обфускація);
- обфускація даних;
- обфускація графа потоку керування.

Лексична обфускація полягає в форматуванні коду програми, зміні його структури таким чином, щоб він став нечитабельним, менш інформативним і важчим для дослідження дизасемблерами і декомпіляторами. Обфускація такого виду включає в себе певні складові:

а) видалення необов'язкових конструкцій мови програмування:

- видалення усіх коментарів в коді або зміна їх на дезінформуючі.

б) символьна обфускація – заміна імен ідентифікаторів (змінних, масивів, структур, функцій і т. п.) на набори символів, які важко сприймати людині. Символьна обфускація найбільш легко реалізується:

- перейменування методів, змінних і т.п. в набір безглузких символів. Наприклад, був метод класу `GetPassword()`, після обфускації даний метод матиме ім'я `KJHS92DSLKaf()`;

- перейменування найменувань змінних і методів у коротші. Проходячи по всіх класах, методах, вони замінюють імена на їх порядкові номери. Наприклад, був метод `GetConnectionString()`, а став `_0()`;

- використання ключових слів мов програмування;

- використання імен, що міняють сенс. Тут швидше діє психологічний чинник. Припустимо, клас `SecurityInformation` з методом `GetInformation()`, а став класом `Car` з методом `Wash()`.

До складніших обфускаторів, які дають більше гарантій того, що наш код не зможуть зрозуміти зловмисники, відносяться обфускатори другого порядку, робота яких пов'язана із

трансформацією структур даних. Вона вважається більш складною, більш сучасною і використовуваною.

Наведемо деякі способи здійснення обфускації даних:

а) розділення змінних. Змінні фіксованого діапазону можуть бути розділені на дві й більше змінних. Для цього змінну V , що має тип x розділяють на k змінних $v_1, \dots, v_k$ типу y тобто $V = v_1, \dots, v_k$.

б) потім створюється набір операцій, що дозволяють витягати змінну типу x зі змінних типу y і записувати змінну типу x у змінні типу y . Як приклад розділення змінних можна розглянути спосіб подання однієї змінної. В логічного типу (boolean) двома змінними b_1 і b_2 типу короткого цілого (short), значення яких буде інтерпретуватися таким чином (табл. 7.1):

Таблиця 7.1–Приклад розділення змінних

B	b_1	b_2
false	0	0
true	0	1
true	1	0
false	1	1

Тоді фрагмент коду (мова C++)

```
bool B ; B = false ; if (B) { ... }
```

буде перетворено у такий:

```
short b1, b2 ;
b1 = b2 = 1 ; // або b1 = b2 = 0
if (!(b1 & b2)) { ... };
```

в) зміна представлення (або кодування). Наприклад, цілочисельну змінну i у нижче представленому фрагменті коду (мова C):

```
...
int i=1;
...
while (i<1000)
{ ... A[i] ...
i++ ;
}
...
```

можна замінити, виразом $i = c_1 \cdot i + c_2$, де c_1, c_2 – константи. В результаті наведений код зміниться й буде складніший для сприйняття:

```

...
int i=11;
int c1=8, c2=3;
...
while (i<8003)
{ ... A[(i-3)/8] ...
  i+=8 ;
}
...

```

г) реструктурування масиву - розділення одного масиву на декілька підмасивів. Наприклад, масив А можна розділити на підмасиви А1 і А2, де масив А1 міститиме парні, а А2 – непарні позиції елементів масиву А.

Тому фрагмент

```

int A[] = {a, b, c, d, e, f};
i = 3;
A[i] = ...;

```

можна замінити на такий:

```

int A1 = {2, 4, 0};
int A2 = {1, 3, 5}; i = 3;
if (($i % 2) == 0) { $A1[$i / 2] = ...; }
else { $A2[$i / 2] = ...; }

```

д) реструктурування масиву – згортання. Наприклад, одномірний масив А розмірністю 6, можна замінити двовимірним масивом В розмірністю 2×3, після чого код

```

int A[] = {1, 2, 3, 4, 5, 6};
for (int i = 0 ; i < 6 ; i++)
{
  A[i] = A[i] + 1;
  print f("%d\n", A[i]);
}

```

можна змінити на такий:

```

int A[2][3] = { {1,2,3}, {4,5,6} };
for (int i = 0 ; i < 2 ; i++)
{
  for (int ii = 0 ; ii < 3 ; ii++)
  { A[i][ii] = A[i][ii] + 1;
    print f("%d\n", A[i][ii]);
  }
}

```

ж) розгортання циклів (loop unrolling). Сутність – тіло циклу розмножується два або більше разів, а умова виходу з циклу і оператор збільшення лічильника відповідним чином модифікуються. Наприклад,

до модифікації:

```
for (i=1; i<=n; i++)
{ // тіло циклу }
після модифікації:
for (i=1; i<=n-1; i++)
{ // тіло циклу }
```

к) розкладання циклів (loop fission). Сутність: цикл зі складним тілом розбивається на декілька окремих циклів з простими тілами й з тим самим простором ітерування:

```
до модифікації:
for (i=1; i<=n; i++)
{ a[i] += c;
  x[i+i]=d+x[i+1]*a[i];
}
після модифікації:
for (i=1; i<=n; i++)
{ a[i] += c; }
for (i=1; i<=n; i++)
{ x[i+i]=d+x[i+1]*a[i]; }
```

л) використання алгебраїчних перетворень для формування неістотного коду у тілі програми. Наприклад:

```
sin2x+cos2x = 1;
cos π = -1;
ln e = 1;
```

Способи реалізації ускладнення логіки – як один з методів захисту від зворотної інженерії застосовується маскуванню програм. Кажучи неформально, маскуванню програми – це таке перетворення її тексту, яке повністю зберігає функціональність, але робить розуміння, зворотну інженерію і модифікацію тексту програми завданням неприйнятно високої вартості. Можна сказати, що маскуванню програм – це обфускація графа потоку керування програмою.

Об'єктами маскуванню є тексти реальних програм, що складаються з сотень функцій по декілька сотень рядків кожна. Замасковані програми повинні вкладатися в обмеження обчислювальної системи, що не може не відбитися на використовуваних методах маскуванню.

Існує декілька груп методів утруднення логіки, які можна комбінувати з іншими методами:

а) маніпулювання функціями:

– відкрита вставка функцій (function inlining) полягає в тому, що тіло функції підставляється в місце її виклику;

- винесення групи операторів (function outlining). Дане перетворення є оберненим до попереднього і добре доповнює його. Деяка група операторів вихідної програми виділяється в окрему функцію. При необхідності створюються формальні параметри;

- усунення бібліотечних викликів (eliminating library calls). Більшість програм мовою Java широко використовують стандартні бібліотеки. Оскільки семантика бібліотечних функцій добре відома, такі виклики можуть дати корисну інформацію при зворотній інженерії програм. Таке перетворення не змінить істотно час виконання програми, але збільшить її розмір;

- переплетення функції (function interleaving). Ідея цього перетворення в тому, що дві або більше функцій поєднуються в одну. Списки параметрів вихідних функцій поєднуються і до них додається ще один параметр, який дозволяє визначити, яка функція в дійсності виконується. Заплутана функція замість операторів if, for і т.д. буде містити оператор switch, розташований всередині нескінченного циклу;

- клонування функцій (function cloning). Можна створити декілька клонів і до кожного з них застосувати різний набір заплутуючих перетворень.

б) Використання непрозорих предикатів:

- основною проблемою при проектуванні перетворень, що заплутують граф потоку керування, є те, як зробити їх не тільки дешевими, але й стійкими. Для забезпечення стійкості більшість перетворень ґрунтується на введенні непрозорих змінних і непрозорих предикатів (opaque predicates);

- сила таких перетворень залежить від складності аналізу непрозорих предикатів і змінних. Аналогічно, предикат Р називається непрозорим, якщо його значення відоме в момент заплутування програми, але важко відновлюване після його завершення.

Використання непрозорих предикатів можна реалізувати по-різному:

- різні способи доступу до елементів масиву. Нехай у програмі створено масив *a*, що ініціалізується заздалегідь відомими значеннями, а далі в програму додаються змінні *i* і *j*, в яких зберігатимуться індекси елементів цього масиву. Тепер непрозорі предикати можуть мати вигляд: $a[i] == a[j]$. Якщо до того ж змінні *i* й *j* у програмі змінюються, то існуючі методи статичного аналізу

дозволять тільки визначити, що i і j вказують на будь-який елемент масиву a . Як простий спосіб можна використати розміщення всіх локальних змінних одного типу в масиві.

– використання покажчиків на спеціально створювані динамічні структури. У цьому підході в програму додаються операції по створенню структур даних (списків, дерев), і додаються операції над покажчиками на ці структури, підібрані таким чином, щоб зберігалися деякі інваріанти, які й використовуються як непрозорі предикати. Один з простих способів: всі значення змінних усіх типів зберігаються в списку, який розміщується в динамічній пам'яті.

– побудова складних булевих виразів за допомогою еквівалентних перетворень із формули true. У найпростішому випадку ми можемо взяти k довільних булевих змінних $x_1 \dots x_k$ і за допомогою еквівалентних алгебраїчних перетворень, коли частина дужок або всі дужки розкриваються, побудувати з них тотожності, наприклад:

$$1 = (x_1 \cup x_1) \cap \dots \cap (x_k \cup x_k); \quad 0 = (x_1 \cup \bar{x}_1) \cap \dots \cap (x_k \cup \bar{x}_k).$$

Це можна використати у багатьох випадках. Так, фрагмент коду:

```
i = 1;
while (i < 101)
{
    ...
    i++;
}
```

після розширення умов циклу матиме вигляд:

```
i = 1;
j = 100;
while ((i < 101) && (j * j * (j + 1) * (j + 1) % 4 == 0) (t))
{
    ...
    i++;
    j = j * i + 3;
}
```

– використання комбінаторних тотожностей. Воно може бути використано для ускладнення логіки програми всюди: і при вбудовуванні функцій, і при клонуванні функцій, і при розгортанні циклів, і при вбудовуванні мертвого та недосяжного кодів тощо.

– внесення недосяжного, мертвого або надлишкового коду. Якщо в програму внесені непрозорі предикати видів PF або PT, гілки умови, які відповідають умові "true" у першому випадку й умові "false" у другому випадку, ніколи не будуть виконуватися.

– недосяжний код (unreachable code) – це фрагмент програми, що ніколи не виконується. Ці гілки можуть бути заповнені довільними

обчисленнями, які схожі на дійсно виконуваний код. Оскільки недосяжний код ніколи не виконується, дане перетворення впливає тільки на розмір заплутаної програми, але не на швидкість її виконання.

– мертвий код (dead code), на відміну від недосяжного, у програмі виконується, але його виконання ніяк не впливає на результат роботи програми. При внесенні мертвого коду розробник повинен бути впевнений, що вставляє фрагмент, який не може впливати на код, що обчислює значення функції.

– надлишковий код (redundant code), на відміну від мертвого коду, виконується, і результат його виконання використовується надалі в програмі, але такий код можна спростити або зовсім видалити, оскільки обчислюється або константне значення, або значення, уже обчислене раніше. Для внесення надлишкового коду можна використати алгебраїчні перетворення ввести в програму математичні тотожності.

7.2 Завдання на лабораторну роботу

Зрозуміти структуру виконуваного файлу, навчитись захищати його від несанкціонованого доступу, позначити найефективніші способи.

7.3 Зміст звіту

7.3.1 Титульний лист, тема і мета роботи.

7.3.2 Відповіді на контрольні питання.

7.3.3 Короткі відомості.

7.3.4 Код програми.

7.3.5 Висновки.

7.4 Контрольні питання

7.4.1 Які групи методів використовують для захисту від несанкціонованого дослідження коду програм? Дайте загальну характеристику цим методам.

7.4.2 Що таке обфускація і які рівні обфускації існують?

7.4.3 Як здійснюється лексична обфускація? З яких етапів вона складається?

7.4.4 Що означає обфускація даних? Які види обфускації даних існують?

7.4.5 Наведіть приклади обфускації даних.

7.4.6 В чому сутність маскуванню програм і яка його мета?

7.4.7 Які заплутуючи перетворення функцій можна здійснити для реалізації захисту програм від дослідження?

7.4.8 Що означає поняття непрозорих предикатів? Де використовуються непрозорі змінні і непрозорі предикати?

7.4.9 Де і як у програмах можна використати комбінаторні тотожності?

7.4.10 Які види внесення неістотного коду ви знаєте? Що таке недосяжний, мертвий та надлишковий коди?

7.5 Порядок виконання роботи

7.5.1 Ознайомитись з основними методами обфускації.

7.5.2 Розробити програми згідно індивідуального завдання. При цьому: перша програма повинна здійснювати захист, не використовуючи при цьому обфускацію, друга програма повинна містити елементи обфускації свого коду.

7.5.3 Дизасемблювати виконуваний код першої програми і здійснити її злам.

7.5.4 Дизасемблювати виконуваний код другої програми і здійснити її злам.

7.5.5 Підготувати звіт щодо ефективного даного виду захисту і складності зламу (табл. 7.2).

Таблиця 7.2 – Результати дослідження

№	Спосіб захисту	Метод обфускації	Мета зламу
1	2	3	4
1	Захист за допомогою серійного ключа.	Реструктурування масивів - згорання	Отримати серійний ключ
2	Захист паролем	Зміна представлення	Отримати пароль

Продовження табл. 7.2 – Результати дослідження

1	2	3	4
3	Захист обмеженням кількості запусків	Використання непрозорих предикатів (комбінаторні тотожності)	Отримати кількість запусків
4	Захист прив'язкою до терміну використання (до дати)	Використання складних булевих виразів	Обійти перевірку дати
5	Захист автентифікацією (логін + пароль)	Реструктурування масиву - розділення	Виділити пароль і (або) логін
6	Захист за допомогою серійного ключа.	Розкладання циклів	Отримати ключ
7	Захист паролем	Шифрування (наприклад, зсув по таблиці ASCII-кодів)	Отримати пароль
8	Захист обмеженням кількості запусків	Використання алгебраїчних перетворень	Змінити кількість запусків
9	Захист прив'язкою до терміну використання (до дати)	Використання способу представлення (наприклад, застосувати структури)	Змінити кількість базову дату
10	Захист автентифікацією (логін + пароль)	Пониження розмірності індексного простору	Виділити пароль і (або) логін

ПЕРЕЛІК ПОСИЛАНЬ

1. Дудатьєв А.В. Захист програмного забезпечення. Ч.1: навчальний посібник /Андрій Дудатьєв, Валентина Каплун, Василь Семеренко – Вінниця: ВНТУ, 2005. – 140 с.
2. Каплун В. А. Захист програмного забезпечення. Ч.2: навчальний посібник /Юрій Баришев, Олександр Дмитришин – Вінниця: ВНТУ, 2013. – 151 с.
3. Войтович О. П. Методичні вказівки до виконання курсового проекту з дисципліни "Захист програмного забезпечення" для студентів напряму підготовки 6.170101 "Безпека інформаційних і комунікаційних систем" /Олеся Войтович, Валентина Каплун – Вінниця: ВНТУ, 2010. – 57 с.

Додаток А

Таблица прости́х чисел

1	2	3	5	7	11	13	17	19	23
29	31	37	41	43	47	53	59	61	67
71	73	79	83	89	97	101	103	107	109
113	127	131	137	139	149	151	157	163	167
173	179	181	191	193	197	199	211	223	227
229	233	239	241	251	257	263	269	271	277
281	283	293	307	311	313	317	331	337	347
349	353	359	367	373	379	383	389	397	401
409	419	421	431	433	439	443	449	457	461
463	467	479	487	491	499	503	509	521	523
541	547	557	563	569	571	577	587	593	599
601	607	613	617	619	631	641	643	647	653
659	661	673	677	683	691	701	709	719	727
733	739	743	751	757	761	769	773	787	797
809	811	821	823	827	829	839	853	857	859
863	877	881	883	887	907	911	919	929	937
941	947	953	967	971	977	983	991	997	1009
1013	1019	1021	1031	1033	1039	1049	1051	1061	1063
1069	1087	1091	1093	1097	1103	1109	1117	1123	1129
1151	1153	1163	1171	1181	1187	1193	1201	1213	1217
1223	1229	1231	1237	1249	1259	1277	1279	1283	1289
1291	1297	1301	1303	1307	1319	1321	1327	1361	1367
1373	1381	1399	1409	1423	1427	1429	1433	1439	1447
1451	1453	1459	1471	1481	1483	1487	1489	1493	1499
1511	1523	1531	1543	1549	1553	1559	1567	1571	1579
1583	1597	1601	1607	1609	1613	1619	1621	1627	1637
1657	1663	1667	1669	1693	1697	1699	1709	1721	1723
1733	1741	1747	1753	1759	1777	1783	1787	1789	1801
1811	1823	1831	1847	1861	1867	1871	1873	1877	1879

Додаток Б

Підключення криптографічної бібліотеки MIRACL

1. Створити новий проєкт «Консольное приложение Win32».
2. Підключити до компіляції каталог ...miracl/include.

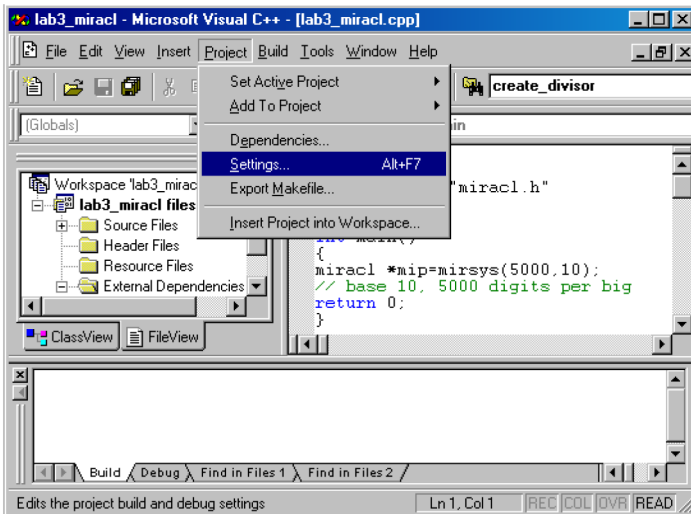


Рисунок Б.1 – Крок 1

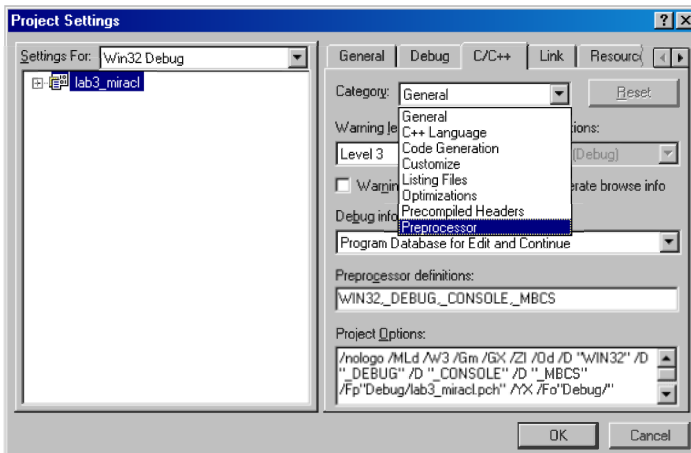


Рисунок Б.2 – Крок 2

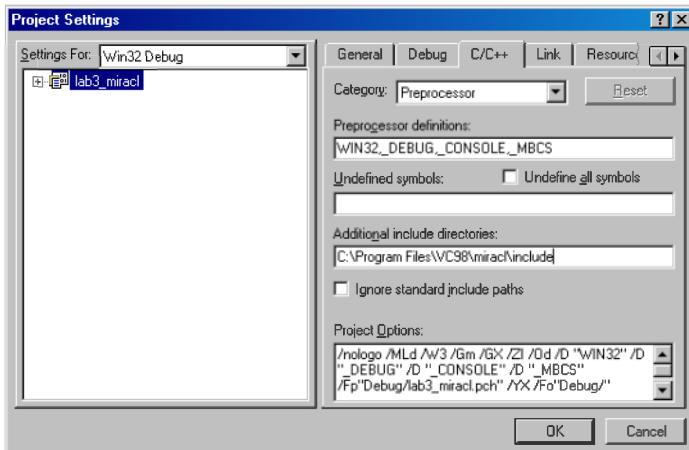


Рисунок Б.3 – Крок 3

3. Додати до проекту файл ...miracle\lib\ms32.lib

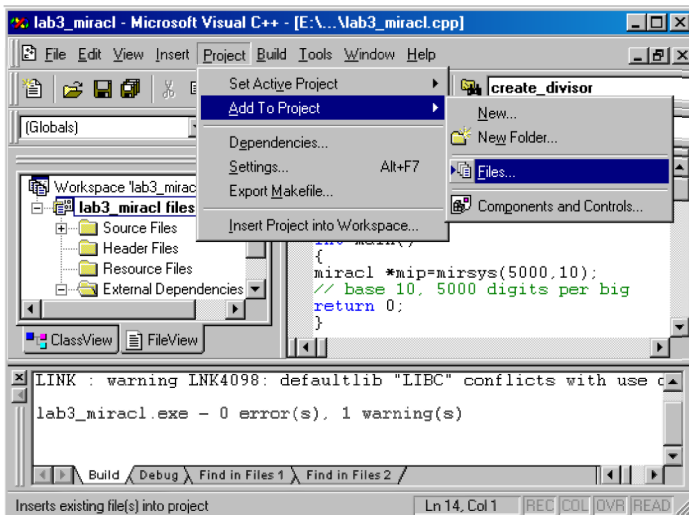


Рисунок Б.4 – Крок 4

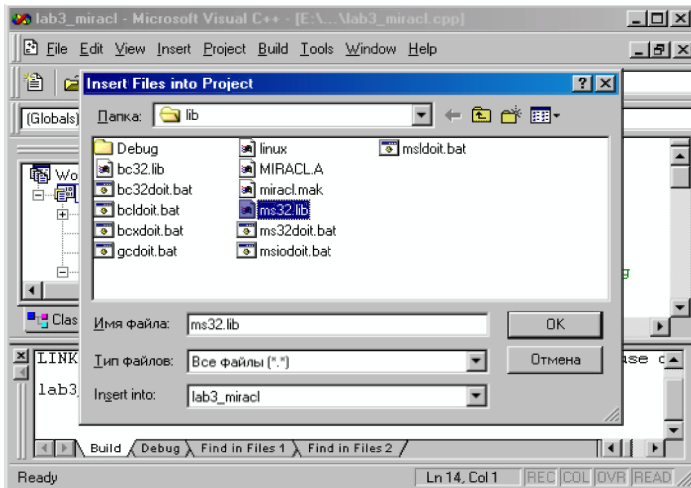


Рисунок Б.5 – Крок 5

4. Набрати програму у вікні редактору.

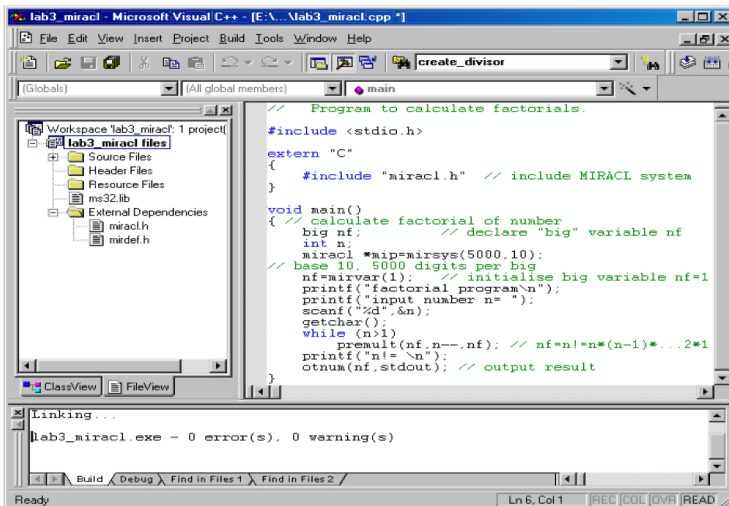


Рисунок Б.6 – Крок 6

5. Запустити проект на компіляцію та виконання.

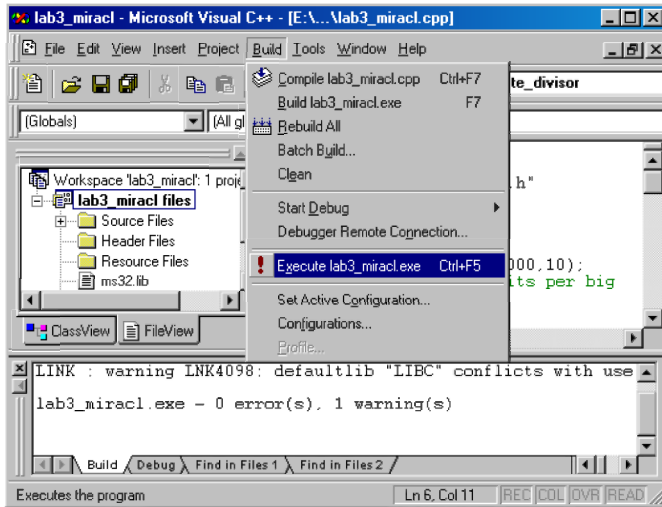


Рисунок Б.7 – Крок 7